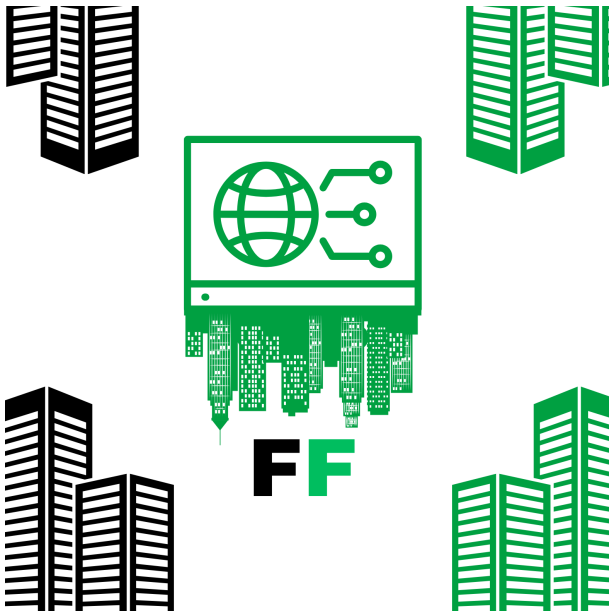
CSIT 321



Team RHEA

Final Report

Table of Contents

Introduction

Project Overview

FeedbackFusion is an intelligent hotel feedback management system designed to help hotels collect, analyse, and derive actionable insights from guest feedback. The system leverages natural language processing (NLP) to automatically analyse sentiment and emotion in customer feedback, identify trends, and generate recommendations for improvement.

The platform centralises feedback from multiple sources, including manual uploads (CSV/JSON files) and automated scraping from review platforms (like Google Reviews). It processes this data through an NLP pipeline to extract sentiment, emotion, and department classification, presenting the results through an intuitive dashboard with visualisation tools and historical analysis capabilities. Built with a modern tech stack including MongoDB, Express.js, React, and Node.js (MERN stack), with Python-based NLP services, FeedbackFusion aims to transform how hotels understand and respond to guest experiences, ultimately improving service quality and guest satisfaction.

Team Information

Name	ID	Role
Abdul Rehman	**7402521**	**Leader**
Junaid Hussain	**7644292**	**Member**
Hanin Elmashtouly	**7866355**	**Member**

Leen Ramadan	**7759927**	**Member**
Youssef Amro	**7581737**	**Scribe**

Purpose

The purpose of FeedbackFusion is to provide hotels with a centralized, data-driven platform that consolidates, analyzes, and delivers actionable insights on guest feedback from multiple sources. By automating the processing of feedback data, the system enables hotel staff to:

- Identify trends in guest sentiment and satisfaction
- Receive alerts when critical issues arise
- Focus resources on areas with the greatest impact on guest experience
- Track the effectiveness of improvement initiatives over time
- Make data-driven decisions based on guest feedback
- Receive personalized, actionable recommendations to enhance guest satisfaction

The platform enhances operational efficiency by reducing the manual effort required to process feedback while improving service quality by ensuring that no critical feedback falls through the cracks. By transforming raw feedback into structured, actionable insights, FeedbackFusion empowers hotels to systematically improve guest experiences and maintain a competitive edge in the hospitality industry.

Scope

FeedbackFusion encompasses several key functional areas:

- Data Collection: Integration of feedback from multiple sources, including file uploads (CSV/JSON) and web scraping of review platforms (implemented for Google Reviews).

- Data Processing: Automated analysis of feedback using NLP techniques to detect sentiment (positive and negative), identify emotions (joy, anger, etc.), and classify feedback by relevant hotel departments.
- Visualization & Insights: Interactive dashboards that present feedback data in meaningful ways, allowing users to filter by department, sentiment, emotion, and date range. The system presents key metrics and identifies trends.
- Alert System: Rule-based alerts triggered when specific conditions are met, such as a threshold of negative reviews for a particular department.
- Historical Analysis: Tools for analyzing feedback data over time, enabling users to track performance trends and measure the impact of changes.
- Recommendation Engine: Rule-based system that suggests actions based on patterns in negative feedback, helping hotels prioritize improvement initiatives.
- User Management: Role-based access control, allowing for different permission levels (Admin, Manager, Supervisor) with appropriate access restrictions.

The system is designed to be accessed through a web-based interface, making it accessible to hotel staff across different roles and locations.

Background Information

Problem Statement

The hotel industry thrives on customer experience, making feedback analysis critical for maintaining guest satisfaction, improving services, and ensuring loyalty. Hotels often receive feedback through various channels, including booking platforms, direct surveys, and social media reviews, creating several challenges:

1. Fragmented Feedback Sources: Hotels receive feedback from multiple channels, making it difficult to track and analyze feedback holistically.
2. Time-Consuming Manual Analysis: Analyzing feedback manually is labor-intensive and time-consuming, often leading to missed trends or actionable insights.
3. Lack of Prioritization: Without a systematic approach, hotels struggle to determine which feedback should take precedence.
4. Limited Historical Insights: Many hotels lack tools to analyze historical feedback trends, limiting their understanding of long-term guest satisfaction patterns.
5. Inconsistent Feedback Quality: The quality of feedback varies significantly across different sources, making it challenging to accurately assess guest sentiment.

6. Lack of Actionable Recommendations: Even when feedback is collected, hotels often lack clear, data-driven recommendations to guide operational improvements.

These challenges can result in delayed responses to guest concerns, missed opportunities for service improvement, and ultimately, decreased guest satisfaction and loyalty.

Market Analysis

The hotel feedback management market has several existing solutions, but most have limitations in providing comprehensive, actionable insights. Hotels increasingly recognize the value of feedback analysis in driving operational improvements and enhancing guest experiences.

Competitor Analysis

Based on our research, we've identified several competitors in the market and analyzed their offerings against our system's features:

Feature	**Our system**	**Medallia**	**TrustYou**	**Zonka Feedback**	**GuestRevu**
Multi-channel feedback collection	yes	yes	basic	yes	yes
Multi-category sentiment analysis	yes	basic	basic	Moderate	basic
Customizable Alerts with rule-based triggers	yes	basic	yes	yes	basic
Advanced historical insights	yes	basic	basic	basic	limited
Role-Specific Insights	yes	no	yes	no	limited
AI-Driven Recommendations	yes	no	no	limited	no

While competitors offer some similar functionalities, FeedbackFusion differentiates itself through:

1. Advanced Sentiment Analysis: Our multi-category sentiment and emotion detection provides more nuanced insights than basic positive/negative classification.
2. Department-Specific Routing: Automatic classification of feedback by department ensures insights reach the right teams.
3. Historical Trend Analysis: Robust tools for analyzing feedback over time help hotels track the impact of service improvements.
4. Customizable Alert System: Rule-based alerts for specific feedback conditions allow hotels to respond quickly to urgent issues.
5. AI-Driven Recommendations: Our system suggests specific actions based on feedback patterns, going beyond just identifying issues.

System Overview and Requirements

Product Perspective

FeedbackFusion operates as a standalone web application with integration capabilities for external data sources. It is designed to fit seamlessly into a hotel's operational framework by improving productivity, reducing manual feedback processing, and enabling more accurate and timely responses to guest issues. The system consists of three main modules:

1. Data Collection Module: Consolidates feedback from multiple sources into a centralized database, including:
    - Manual uploads of CSV/JSON files
    - Automated scraping of review sites (implemented for Google Reviews)
1. Data Processing Module: Processes raw feedback through NLP pipelines to:
    - Analyze sentiment (positive and negative)
    - Detect emotions (joy, anger, sadness, neutral, disgust, surprise & fear)
    - Classify feedback by department (Front Desk, Room Service, Housekeeping, Food & Beverage, Maintenance & Amenities, Restaurant/Café & General)
    - Generate rule-based recommendations
1. Data Visualization Module: Presents processed data through interactive dashboards that enable:
    - Filtering by: keywords, department, sentiment, emotion, and time period.

- Viewing historical trends and patterns
- Accessing actionable insights and recommendations

While FeedbackFusion is primarily a standalone system, it is designed with APIs and integration points that allow for potential connections with hotel Property Management Systems (PMS), Customer Relationship Management (CRM) tools, and other hospitality software solutions.

User Classes and Characteristics

FeedbackFusion supports three primary user roles, each with specific permissions and access levels:

1. Admin:
    - Full system access
    - User management (add, remove, grant permissions)
    - Upload datasets
    - Generate reports
    - Set up alerts
    - Access all analytics and historical data
1. Manager:
    - Department-specific view of insights and analytics
    - Generate reports for their department
    - Set up alerts for their department
    - Access historical analysis for their department
1. Supervisor:
    - Limited access to insights relevant to their supervised area
    - View results filtered to their department
    - Receive alerts for their specific area of responsibility
    - Basic search and filtering capabilities

The table below summarizes the feature access by role:

Feature	Admin	Manager	Supervisor
Login/Logout	✓	✓	✓
Manage Users	✓		
Upload New Datasets	✓	✓	✓
Create Alerts	✓	✓	
Receive Alerts	✓	✓	✓
View Results	✓	✓	✓
Filter Categories	✓	✓	✓
Search Terms	✓	✓	✓
Historical Analysis	✓	✓	✓

Operating Environment

FeedbackFusion is designed to operate in a modern web environment with the following specifications:
Server Environment:

- Cloud-based hosting (AWS, Google Cloud, or similar)
- Node.js runtime for the backend API
- MongoDB for database storage
- Python environment for NLP services

Client Environment:

- Modern web browsers (Chrome, Firefox, Safari, Edge)

- Responsive design supporting desktop, laptop, and tablet devices
- Minimum screen resolution of 1024x768

Network Requirements:

- Stable internet connection with minimum 1 Mbps bandwidth
- HTTPS protocol for secure data transmission
- WebSocket support for real-time notifications (planned feature)

Design and Implementation Constraints

Several constraints influenced the design and implementation of FeedbackFusion:

1. The system must handle potentially large volumes of feedback data while maintaining responsive performance. This requires efficient data processing algorithms and potentially asynchronous processing for time-intensive operations.
2. Guest feedback may contain sensitive information, requiring strict adherence to data protection regulations such as GDPR. This necessitates secure storage, encrypted transmission, and appropriate access controls.
3. The system must provide comprehensive features while maintaining an intuitive user interface. This requires careful organization of functionality and progressive disclosure of advanced features.
4. The database design and application architecture must accommodate growing volumes of data as hotels collect more feedback over time, without degradation in performance.
5. External APIs for review sites and booking platforms may have rate limits or access restrictions that impact the frequency and volume of data collection.
6. The accuracy of sentiment analysis and emotion detection depends on the quality of training data and model selection. The system must account for potential misclassifications and provide mechanisms for correction.

Assumptions and Dependencies

FeedbackFusion operates under several key assumptions and dependencies:

Assumptions:

1. Hotels have access to sufficient guest feedback data from various channels to make the system valuable.
2. Users have basic proficiency with web-based applications and data interpretation.
3. Hotel staff will regularly check and respond to insights and alerts generated by the system.

Dependencies:

1. The system depends on access to external data sources through APIs or web scraping capabilities.
2. The accuracy of insights depends on the quality and training of natural language processing models.
3. Reliable operation depends on cloud services for hosting, storage, and computing resources.
4. Consistent internet connectivity is required for real-time data access and processing.
5. The frontend interface requires modern web browsers that support current JavaScript and CSS standards.

System Architecture

Architectural Design

FeedbackFusion employs a modular, layered architecture partitioned into several high-level subsystems. Each subsystem fulfills a specific responsibility, with clear interconnections to achieve full system functionality.

The architecture consists of the following key layers:

1. Data Collection Layer: Responsible for gathering feedback from various sources, including:
    - File upload module for CSV/JSON data
    - Web scraping module for external review sites
    - API integration module for direct connections to external platforms
1. Core Processing Layer: Handles data processing and analysis, including:
    - Preprocessing module for data cleaning and normalization
    - NLP service integration for sentiment and emotion analysis

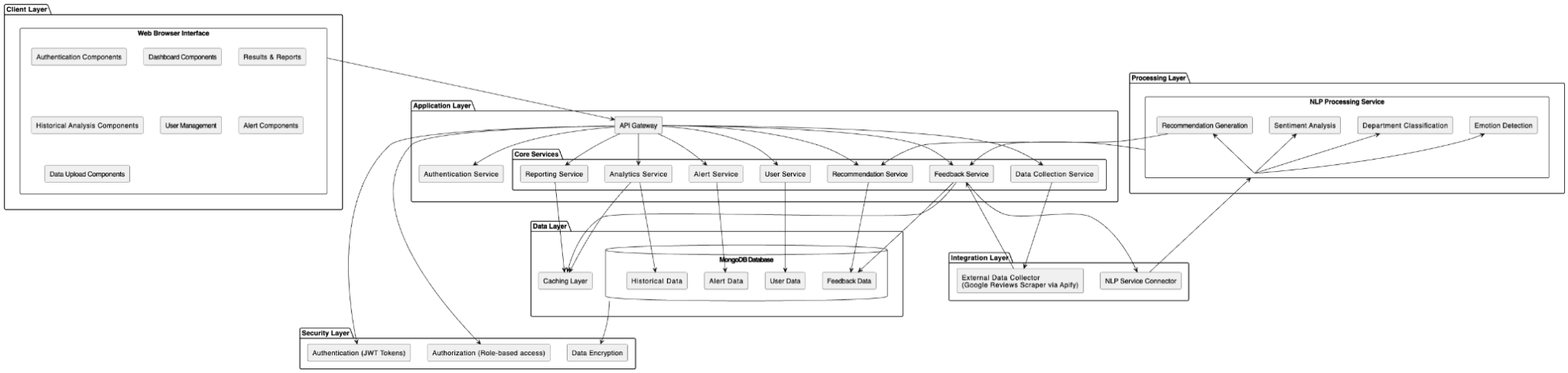
    - Department classification module
    - Recommendation generation module

1. Storage Layer: Manages persistent data storage, including:
    - MongoDB database for feedback, user, and system data
    - Caching mechanisms for performance optimization
    - Data backup and recovery systems

1. API Layer: Provides standardized interfaces for:
    - Frontend-backend communication
    - External service integration
    - Future third-party integrations

1. Visualization Layer: Delivers the user interface, including:
    - Dashboard components
    - Interactive charts and graphs
    - Filtering and search capabilities
    - Report generation tools

1. Security Layer: Spans across all other layers, providing:
    - Authentication and authorization
    - Data encryption
    - Access control



This layered approach enables modularity, making it easier to develop, test, and maintain individual components while ensuring they work together seamlessly.

Decomposition Description

The FeedbackFusion system is decomposed into several interconnected modules, each responsible for specific functionality:

1. User Management Module:
    - Handles user authentication and authorization
    - Manages user roles and permissions
    - Enforces access control policies across the system
    - Provides user profile management capabilities
1. Data Collection Module:
    - File Upload Component: Processes CSV and JSON files containing feedback data
    - Apify Scraper Integration: Automates collection of Google Reviews using the Apify service

    - Data Validation Component: Ensures data quality and consistency before storage
1. Feedback Processing Module:
    - NLP Service Integration: Connects to Python-based sentiment and emotion analysis services
    - Department Classification: Automatically assigns feedback to relevant hotel departments
    - Deduplication Logic: Prevents duplicate entries from multiple sources
1. Analytics Module:
    - Statistical Analysis Component: Calculates key metrics from feedback data
    - Trend Detection: Identifies patterns and changes over time
    - Visualization Generation: Prepares data for dashboard display
1. Alerts System:
    - Alert Definition Component: Allows users to set up custom alert criteria (department, threshold & priority)
    - Monitoring Service: Continuously checks for conditions that trigger alerts
    - Notification Component: Delivers alerts to appropriate users
1. Recommendation Engine:
    - Pattern Recognition Component: Identifies recurring issues in negative feedback
    - Rule-Based Suggestion System: Maps issues to predefined recommendations
    - Priority Assignment: Ranks recommendations by potential impact

Design Rationale

The architecture was selected to support FeedbackFusion's primary goals of scalability, modularity, and efficient processing. A layered approach allows each function to operate independently while facilitating communication across subsystems. Key considerations include:

- Modularity: Allows each feature, such as sentiment analysis and alerts, to be developed and tested independently.
- Scalability: A modular design ensures the platform can accommodate growing data volumes and additional feedback channels.
- Security and Compliance: The Security Layer is integrated across all modules to protect data privacy, implement role-based access, and comply with data protection regulations.

Alternative architectural approaches were considered during the design phase:

1. Monolithic vs. Microservices: We opted for a modular monolith rather than full microservices architecture, striking a balance between separation of concerns and development simplicity. This approach provides many benefits of microservices while avoiding the operational complexity of distributed systems.
2. Database Selection: We evaluated both SQL (PostgreSQL) and NoSQL (MongoDB) options, ultimately selecting MongoDB for its flexibility with unstructured data and schema evolution, which is valuable for a system processing varied feedback formats.
3. NLP Implementation: We considered both embedding NLP functionality directly within the application versus creating a separate service. The separate service approach was chosen to allow for independent scaling and technology flexibility (Python-based NLP models).

High-Level System Operation

The FeedbackFusion system operates through the following sequence of processes:

1. Data Collection:
    - Feedback is gathered from multiple sources:
        - Manual uploads of CSV/JSON files by hotel staff
        - Automated scraping of Google Reviews via Apify integration
    - Raw feedback data is temporarily stored for processing
1. Data Processing:
    - Collected feedback undergoes preprocessing:
        - Validation and cleaning to ensure data integrity
        - Format normalization to standardize diverse sources
        - Deduplication to prevent repeated entries
    - The cleaned data is then analyzed:
        - Sent to the NLP service for sentiment and emotion analysis
        - Processed for department classification
        - Evaluated for potential alert triggering

1. Sentiment and Insight Extraction:
    - The NLP service performs:
        - Sentiment classification (positive & negative)
        - Emotion detection (joy, anger, sadness, etc.)
        - Keyword extraction for topic identification
    - Results are stored in the database with the original feedback
1. Visualization & Reporting:
    - Processed feedback data is organized for display:
        - Aggregated metrics (average ratings, weighted sentiment score)
        - Time-based trends (changes in sentiment over time)
        - Departmental breakdowns (sentiment performance by hotel area)
    - Users can filter and explore data through the dashboard
1. Alerts & Notifications:
    - The system monitors incoming feedback against defined alert criteria
    - When thresholds are exceeded (e.g., multiple negative reviews for a department):
        - An alert is created in the system
        - Relevant users are notified based on their role and department
    - Users can view and manage alerts through the dashboard
1. Recommendation Generation:
    - Patterns in negative feedback are analyzed against predefined rules
    - When recurring issues are identified:
        - The system suggests potential improvements
        - The system counts the number of negative keywords, and displays that to the user as: high, medium or low priority
        - Recommendations are presented with relevant feedback examples
    - Hotel management can check of the recommendation once implemented

Data Flow

The data within FeedbackFusion flows through the system in the following manner:

1. Input Sources → Data Collection Module:
    - User-uploaded files (CSV/JSON)
    - Scraped review data (Google Reviews via Apify)
1. Data Collection Module → Storage Layer:
    - Raw feedback data is temporarily stored
    - Metadata about source, time, rating, feedback text, and upload user is recorded
1. Storage Layer → Processing Module:
    - Raw feedback is retrieved for processing
    - Preprocessing cleans and normalizes the data
1. Processing Module → NLP Service:
    - Preprocessed text is sent to external NLP service
    - Sentiment, emotion, and department classification is performed
1. NLP Service → Processing Module:
    - Analysis results are returned
    - Additional processing occurs if the sentiment is negative (e.g., recommendation matching)
1. Processing Module → Storage Layer:
    - Processed feedback with analysis results is stored
    - Historical data is updated
1. Storage Layer → Analytics Module:
    - Feedback data is retrieved for analysis
    - Statistical calculations and trend detection are performed
1. Analytics Module → Visualization Layer:
    - Processed analytics data is formatted for visualization
    - Dashboard components receive data for display
1. Storage Layer → Alert System:
    - New feedback is checked against alert criteria
    - Matching conditions trigger alert creation
1. Visualization Layer → User Interface:
    - Visualized data, alerts, and recommendations are presented to users
    - Users interact with the interface to explore and manage data

This data flow ensures that feedback is properly collected, processed, analyzed, and presented to users, enabling them to derive actionable insights and improve hotel operations.

Software and Hardware Requirements

Development Tools and Platforms

Programming Languages:

- JavaScript/Node.js: Core language for backend development
- JavaScript/React: Frontend development
- Python: NLP services and sentiment analysis

Frameworks and Libraries:

- Backend:
    - Express.js: Web application framework
    - Mongoose: MongoDB object modeling
    - Node-cron: Scheduled tasks
    - Multer: File upload handling
    - Apify Client: Integration with web scraping service
- Frontend:
    - React: UI library
    - React Router: Navigation
    - Recharts: Data visualization
    - Tailwind CSS: Styling
    - Framer Motion: Animations
    - jsPDF & XLSX: Report generation

- NLP & Data Processing:
    - Flask: API framework for NLP service
    - Transformers: NLP models for sentiment analysis
    - NLTK: Natural language processing tools
    - Scikit-learn: Machine learning utilities

Database:

- MongoDB: NoSQL database for storing feedback, user data, and system configuration

Version Control:

- Git: Source code management
- GitHub: Collaboration and code hosting

Development Tools:

- Visual Studio Code: Primary IDE
- Postman: API testing
- MongoDB Compass: Database management

Hardware Requirements

Development Environment:

- Modern computer with multi-core processor (Intel i5/i7 or equivalent)
- Minimum 16GB RAM (32GB recommended for NLP development)
- SSD storage for better performance
- High-resolution display for UI development
- Stable internet connection

Production Environment:

- Server Requirements:
    - Multi-core processor (e.g., Intel Xeon or AMD EPYC)
    - Minimum 16GB RAM, 32GB recommended for larger deployments
    - SSD storage with at least 100GB capacity, expandable based on data volume
    - Gigabit Ethernet for network connectivity
- Client Requirements:
    - Any modern device capable of running supported web browsers
    - Minimum 4GB RAM
    - Display resolution of at least 1024x768
    - Standard input devices (keyboard, mouse/touchpad)
- Network Infrastructure:
    - High-speed internet connection (minimum 10 Mbps)
    - Firewall for security
    - Optional: VPN for secure remote access

System Setup Requirements

Server Setup:

- Operating System: Linux (Ubuntu/Debian recommended), Windows Server, or macOS
- Node.js runtime environment (v14+)
- MongoDB instance (v4.4+)
- Python environment (v3.8+) for NLP services
- HTTPS certificate for secure connections
- Backup solution for database and file storage

Deployment Options:

- Self-hosted:

    - Dedicated or virtual server with required specifications
    - Container setup with Docker (optional but recommended)
    - Regular maintenance and updates
- Cloud Deployment (Recommended):
    - AWS, Google Cloud, or Azure platform
    - MongoDB Atlas for database (or equivalent managed service)
    - Automated scaling and backup solutions
    - Content Delivery Network for global performance

Client Setup:

- Modern web browser (Chrome, Firefox, Safari, Edge latest versions)
- JavaScript enabled
- Cookies enabled for session management
- Minimum screen resolution: 1024x768 (responsive design supports various screen sizes)

External Interface Requirements

User Interfaces:

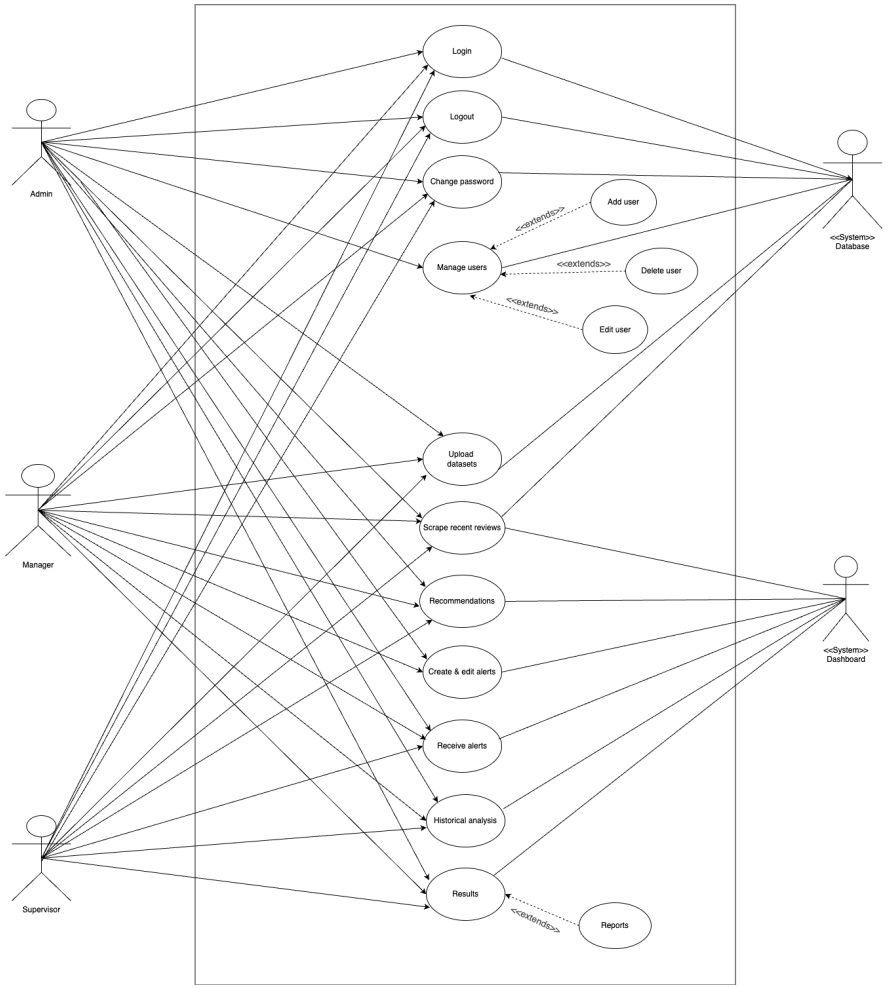
1. Login/Authentication Interface:
    - Username and password fields
    - Role-based access control
    - Session management
    - Password reset functionality
1. Dashboard Interface:
    - Overview of key metrics and recent activity

    - Quick access to main features
    - Notifications for alerts and system updates
    - Role-specific views based on user permissions
1. Data Management Interface:
    - File upload functionality
    - Processing status monitoring
    - Error handling and reporting
1. Analysis Interface:
    - Interactive charts and graphs
    - Filtering controls
    - Time period selection
    - Department and category selection
    - Search functionality
1. Administration Interface:
    - User management
    - System settings
    - Audit logging

All interfaces follow responsive design principles to ensure usability across devices with different screen sizes.

System Features and Components

Use Case Diagram



Use Case Descriptions

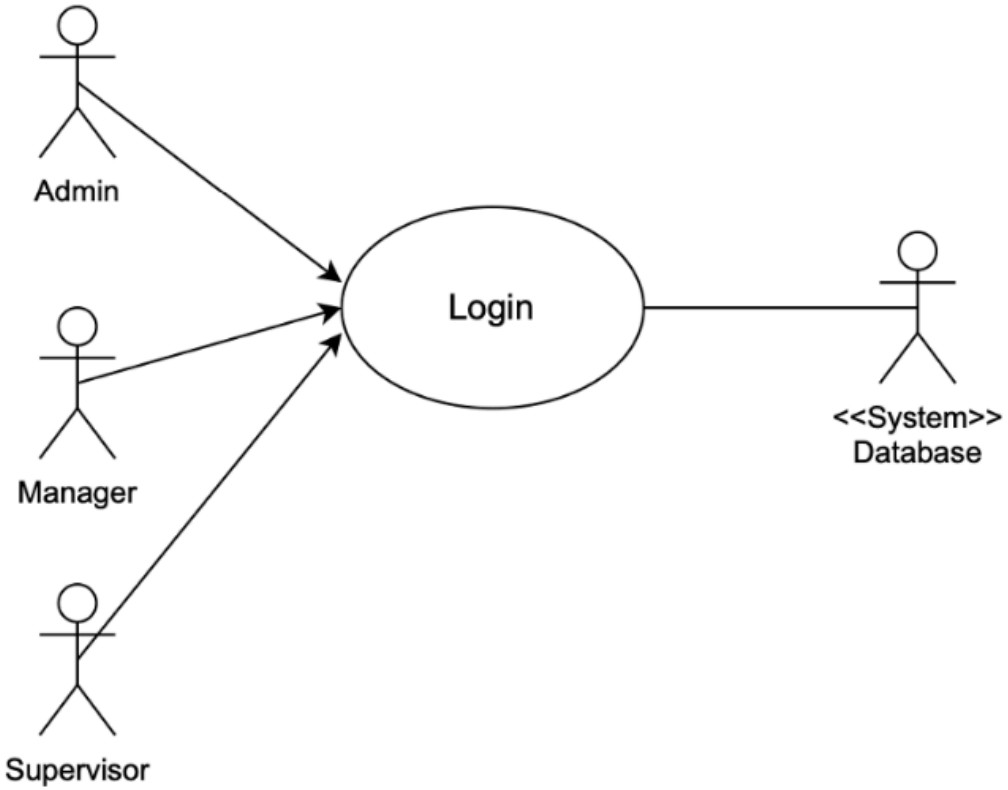
Login

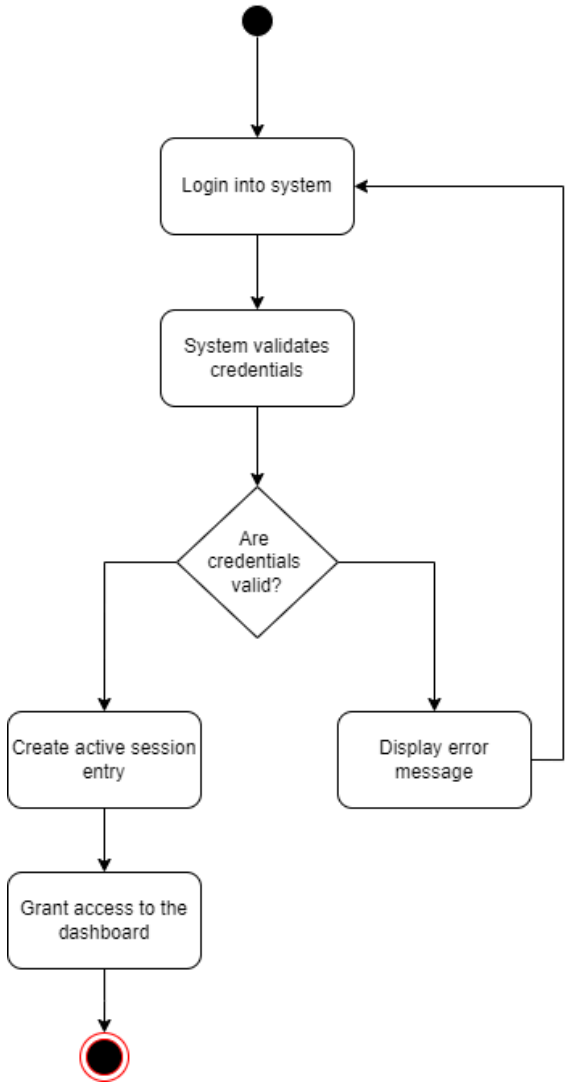
Goal: Allow users to securely authenticate and access the system
Actors: Admin, Manager, Supervisor
Preconditions: User has valid credentials
Main Flow:

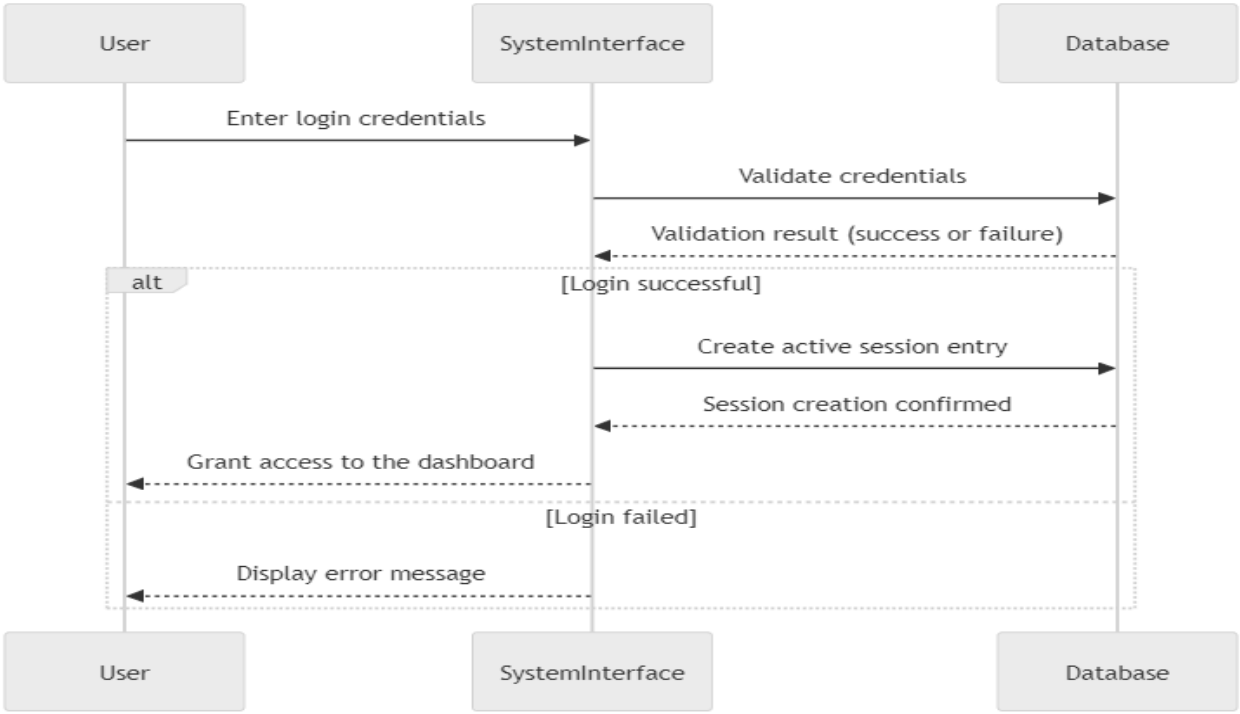
1. User navigates to the login page
2. User enters username and password
3. System verifies credentials against the database
4. System creates a secure session and grants appropriate access based on user role

Alternative Flows:

- Invalid credentials: System displays an error message
- Forgotten password: User can request password reset
- Postconditions: User is authenticated and can access authorized features


Admin
Manager
Supervisor
Login
<<System>>
Database


Login into system
System validates credentials
Are credentials valid?
Create active session entry
Display error message
Grant access to the dashboard


User
SystemInterface
Database
Enter login credentials
Validate credentials
Validation result (success or failure)
alt
[Login successful]
Create active session entry
Session creation confirmed
Grant access to the dashboard
[Login failed]
Display error message
User
SystemInterface
Database

Manage Users

Goal:

Allow the Admin to manage users within the system through various operations such as adding users, removing users, and edit users.

Actors:

Admin

Preconditions:

- Admin is authenticated and has the necessary permissions to manage users.

Main Flow:

1. Admin accesses the **Manage Users** section through the system interface.
2. Admin selects from one of the following operations:
    - **Add User**
    - **Remove User**
    - **Edit User**
3. Based on the selection, the system performs the necessary validation and actions.
4. System updates the database with any changes.
5. System provides a confirmation of the action taken.

Extensions:

- **Add User:**
    - Admin enters the required details for the new user.
    - System validates the information.
    - System creates the user account and stores it in the database.
    - **Output:** Confirmation of successful account creation.
- **Remove User:**
    - Admin selects the user to be removed.

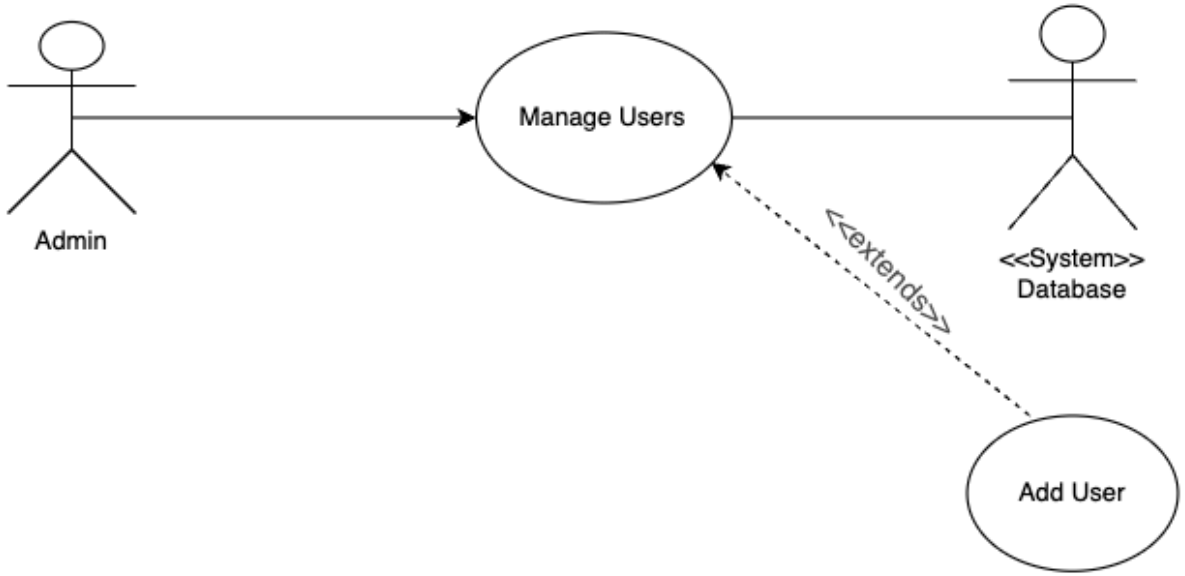
    - System deletes the user from the database and logs the action.
    - **Output:** Confirmation of successful user removal.
- **Edit User:**
    - Admin selects the user and assigns a specific role or modifies permissions.
    - System updates the role and permissions in the database.
    - **Output:** Confirmation of successful role or permission update.

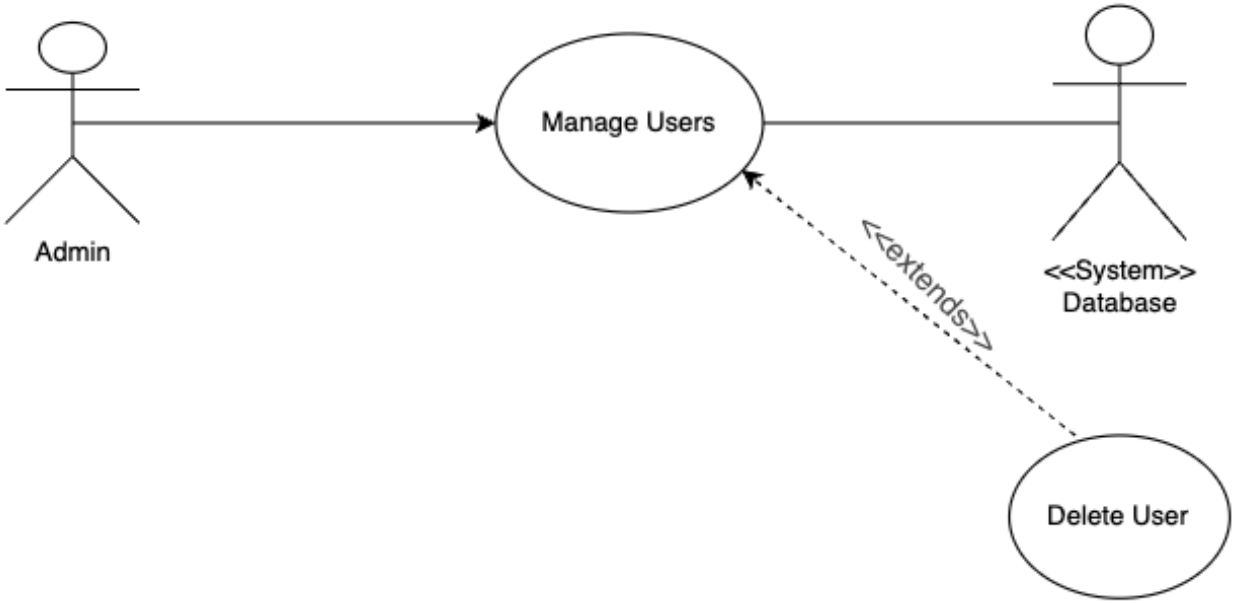
Exceptions:

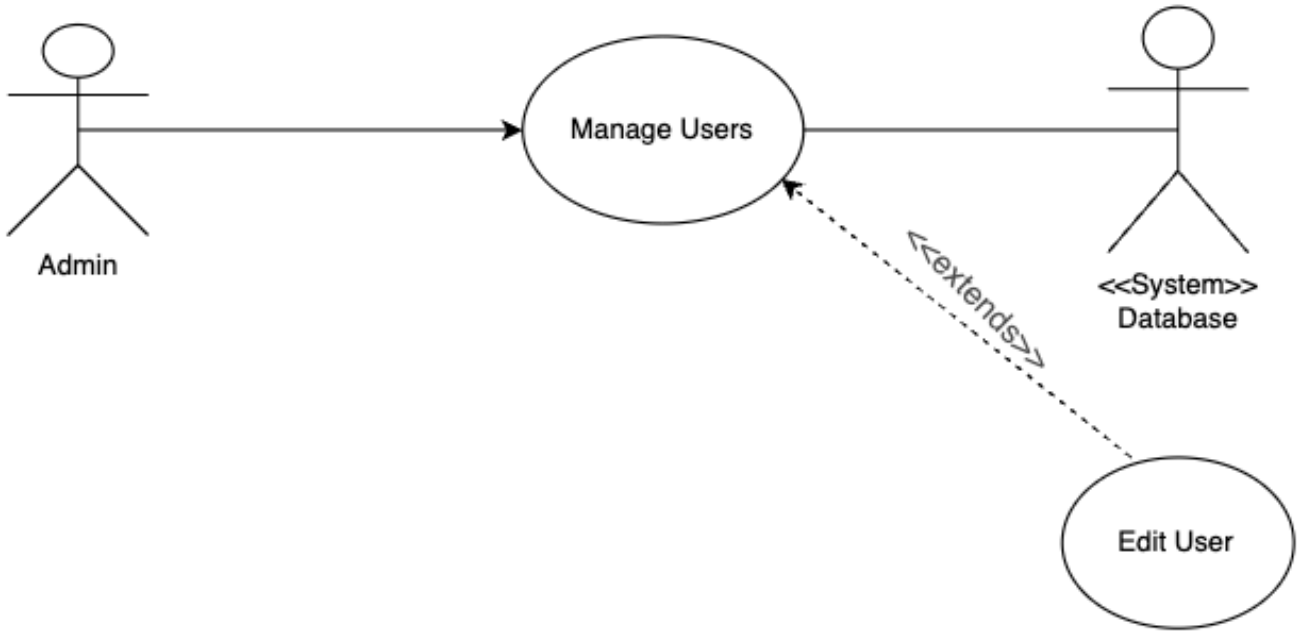
- **Add User Exceptions:**
    - **Duplicate Account:** System displays an error if a user account with the same email already exists.
    - **Invalid Data Format:** System shows an error if the provided information doesn't meet the required
- **Grant Roles and Permissions Exceptions:**
    - **Permission Conflict:** System rejects the action and notifies the admin if there is a role or permission conflict.

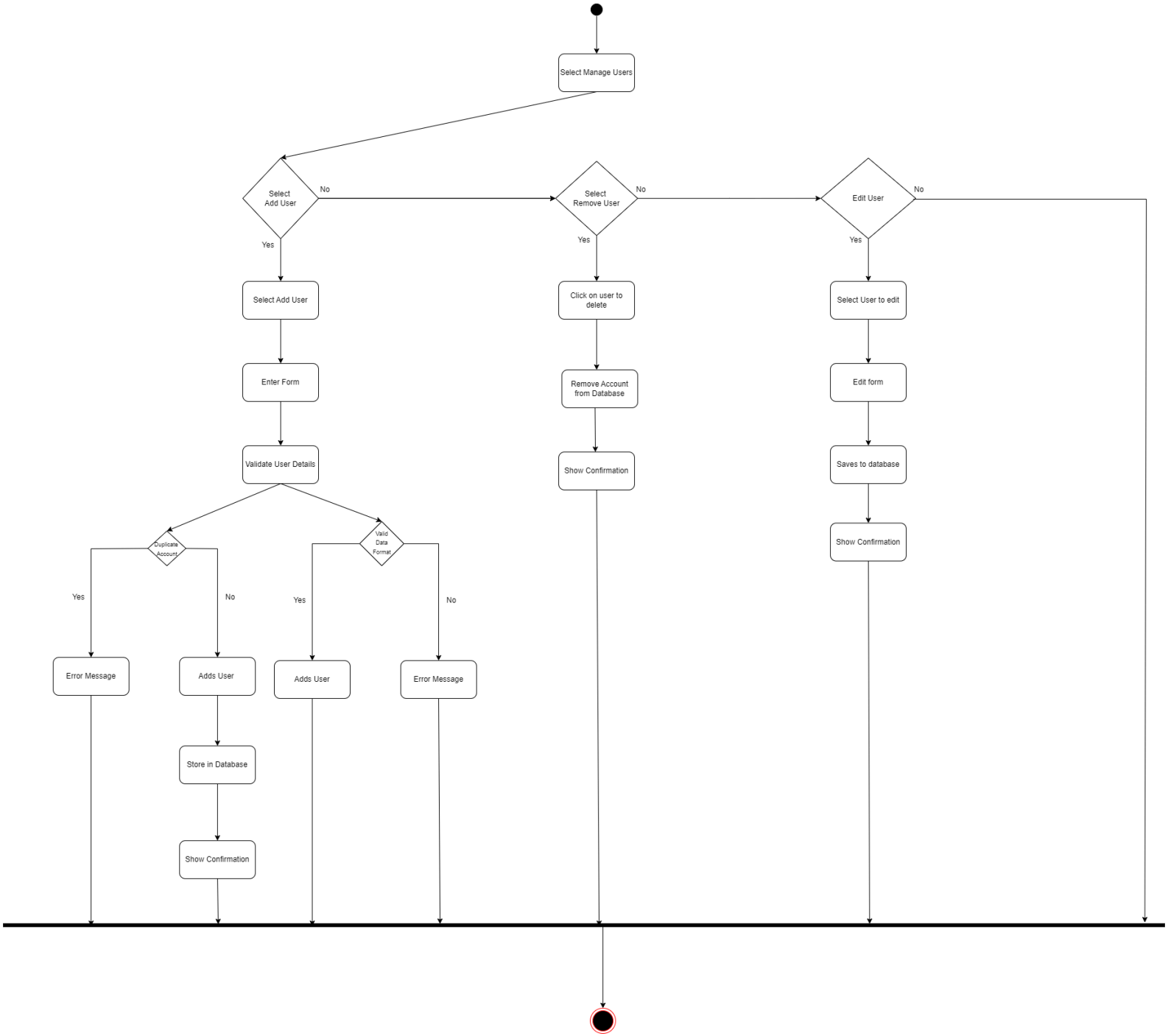
Postconditions:

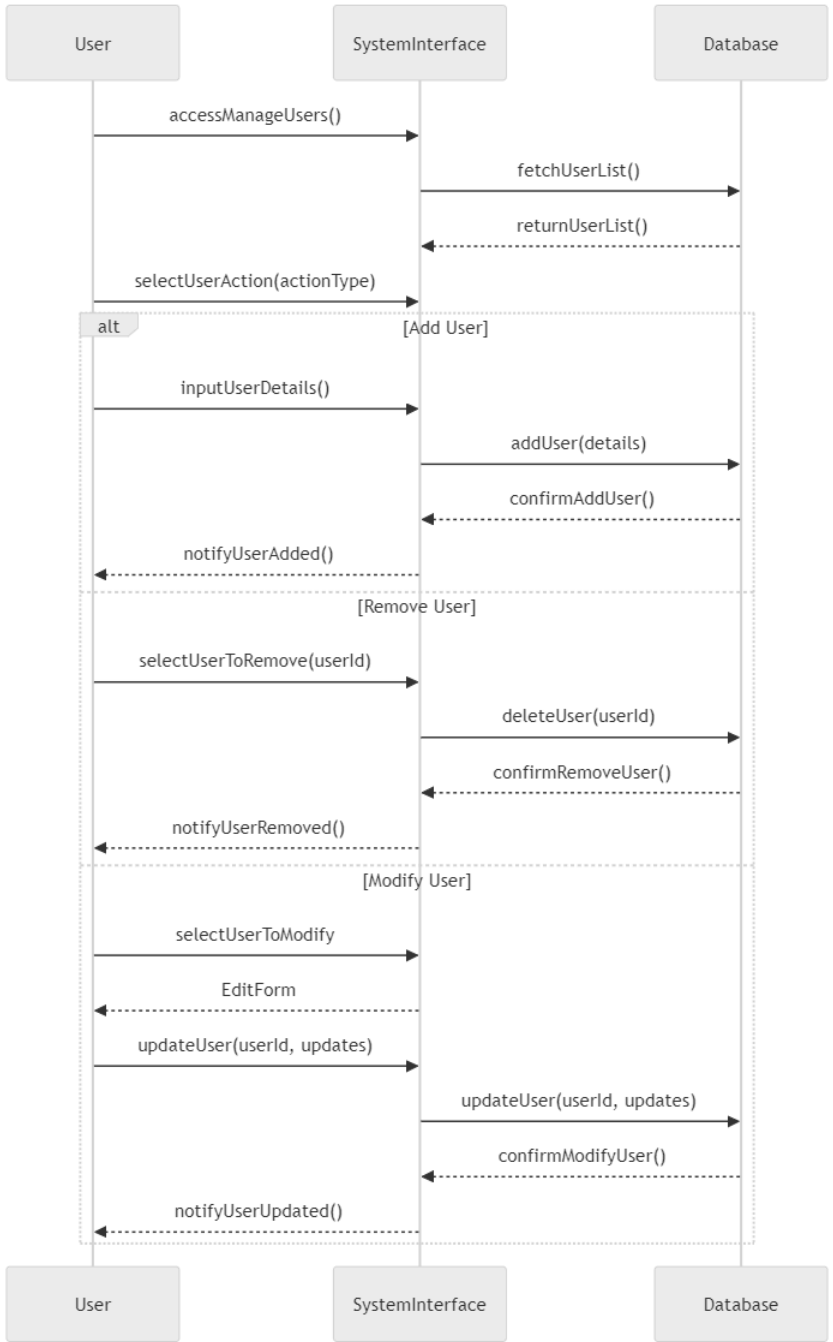
- User information in the database is updated based on the selected action.
- System logs the activity for auditing purposes.


Admin
Manage Users
<<System>>
Database
<<extends>>
Add User


Admin
Manage Users
<<System>>
Database
<<extends>>
Delete User


Admin
Manage Users
<<System>>
Database
<<extends>>
Edit User


Select Manage Users
Select Add User
No
Select Remove User
No
Edit User
No
Yes
Yes
Yes
Select Add User
Click on user to delete
Select User to edit
Enter Form
Remove Account from Database
Edit form
Validate User Details
Show Confirmation
Saves to database
Show Confirmation
Duplicate Account
Valid Data Format
Yes
No
Yes
No
Error Message
Adds User
Adds User
Error Message
Store in Database
Show Confirmation


User
SystemInterface
Database
accessManageUsers()
fetchUserList()
returnUserList()
selectUserAction(actionType)
alt
[Add User]
inputUserDetails()
addUser(details)
confirmAddUser()
notifyUserAdded()
[Remove User]
selectUserToRemove(userId)
deleteUser(userId)
confirmRemoveUser()
notifyUserRemoved()
[Modify User]
selectUserToModify
EditForm
updateUser(userId, updates)
updateUser(userId, updates)
confirmModifyUser()
notifyUserUpdated()
User
SystemInterface
Database

Upload Datasets

Goal: Allow users to upload feedback data files for processing

Actors: Admin, Manager, Supervisor

Preconditions: User has appropriate permissions and valid data files

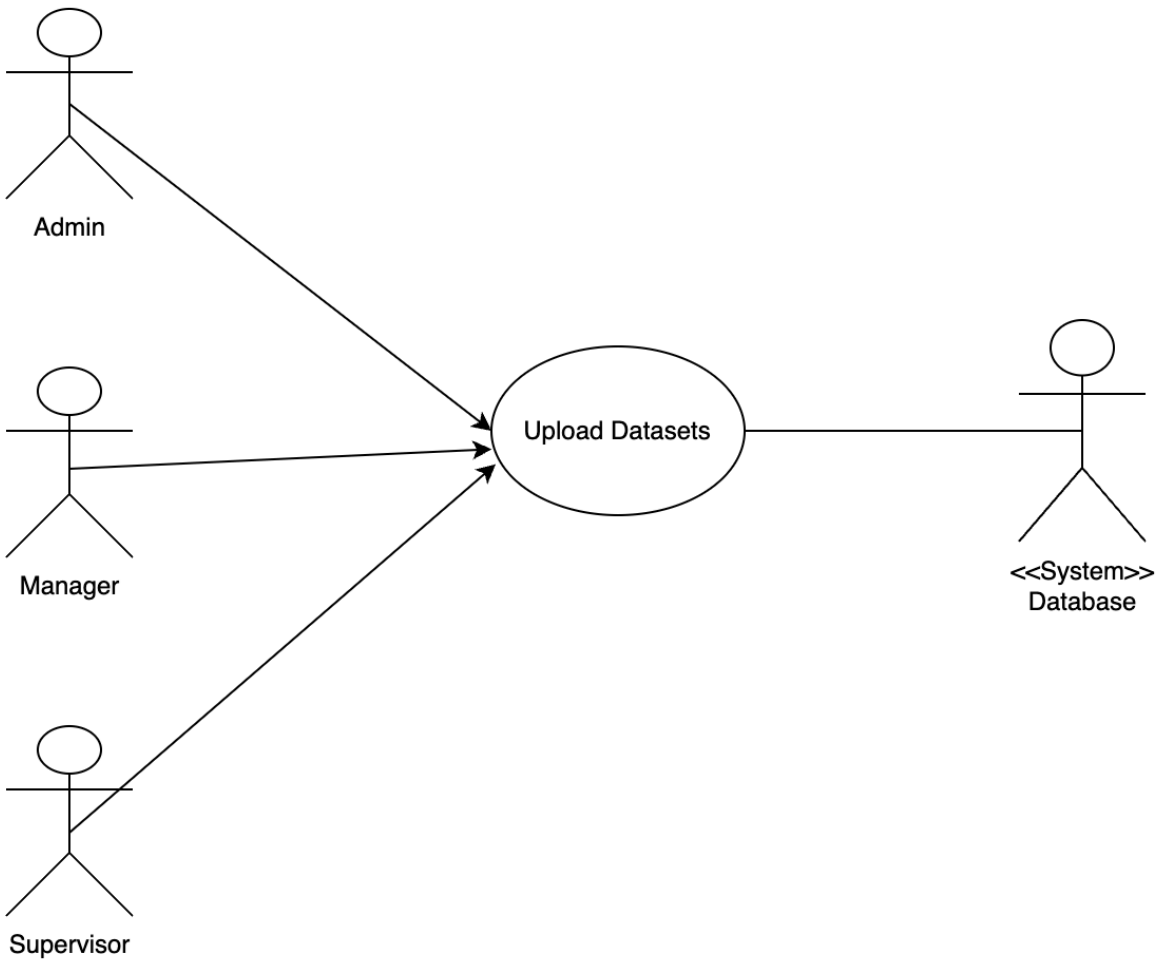
Main Flow:

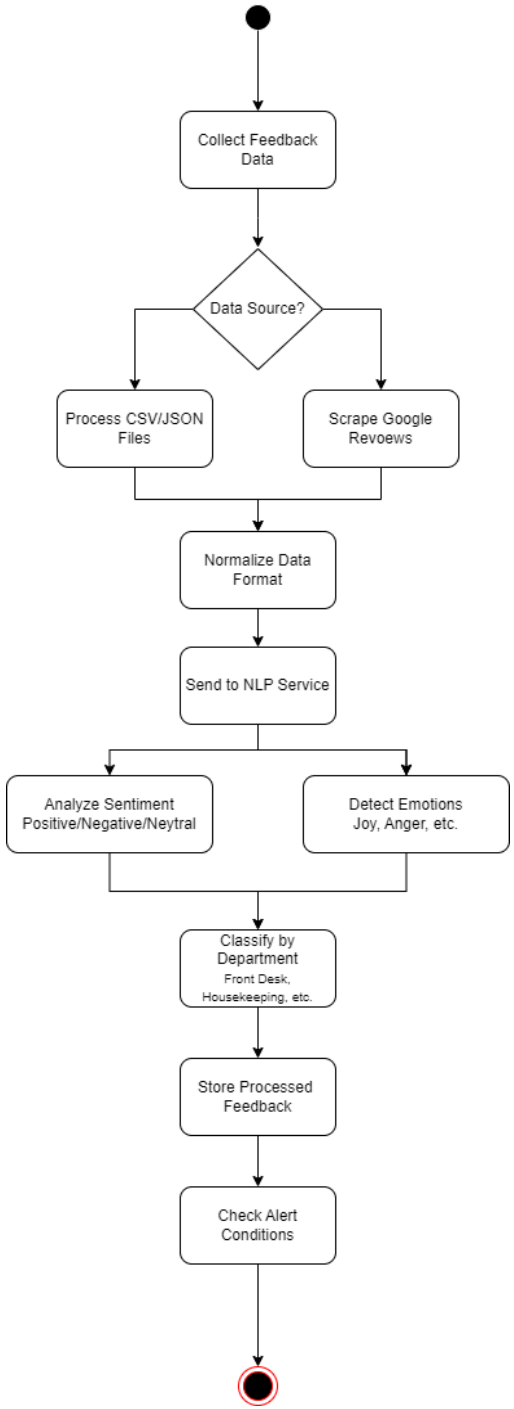
1. User navigates to the data upload section
2. User selects a CSV or JSON file containing feedback data
3. System validates the file format and structure
4. System processes and stores the feedback data
5. System displays a confirmation of successful upload

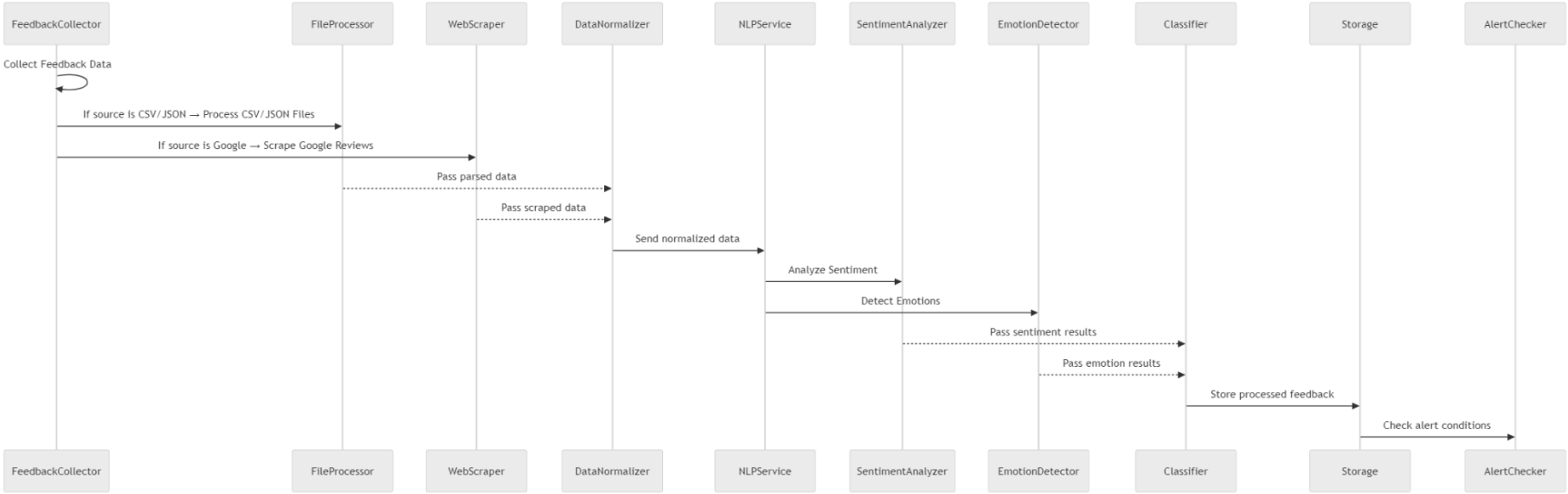
Alternative Flows:

- Invalid file format: System displays an error message
- Duplicate data: System identifies and handles duplicate entries

Postconditions: Feedback data is stored in the database and ready for analysis


Admin
Manager
Supervisor
Upload Datasets
<<System>>
Database


Collect Feedback Data
Data Source?
Process CSV/JSON Files
Scrape Google Revoews
Normalize Data Format
Send to NLP Service
Analyze Sentiment Positive/Negative/Neytral
Detect Emotions Joy, Anger, etc.
Classify by Department
Front Desk, Housekeeping, etc.
Store Processed Feedback
Check Alert Conditions


FeedbackCollector
FileProcessor
WebScraper
DataNormalizer
NLPService
SentimentAnalyzer
EmotionDetector
Classifier
Storage
AlertChecker
Collect Feedback Data
If source is CSV/JSON → Process CSV/JSON Files
If source is Google → Scrape Google Reviews
Pass parsed data
Pass scraped data
Send normalized data
Analyze Sentiment
Detect Emotions
Pass sentiment results
Pass emotion results
Store processed feedback
Check alert conditions
FeedbackCollector
FileProcessor
WebScraper
DataNormalizer
NLPService
SentimentAnalyzer
EmotionDetector
Classifier
Storage
AlertChecker

Scrape Reviews

Goal: Automatically collect feedback from external review platforms

Actors: Admin, Manager, Supervisor

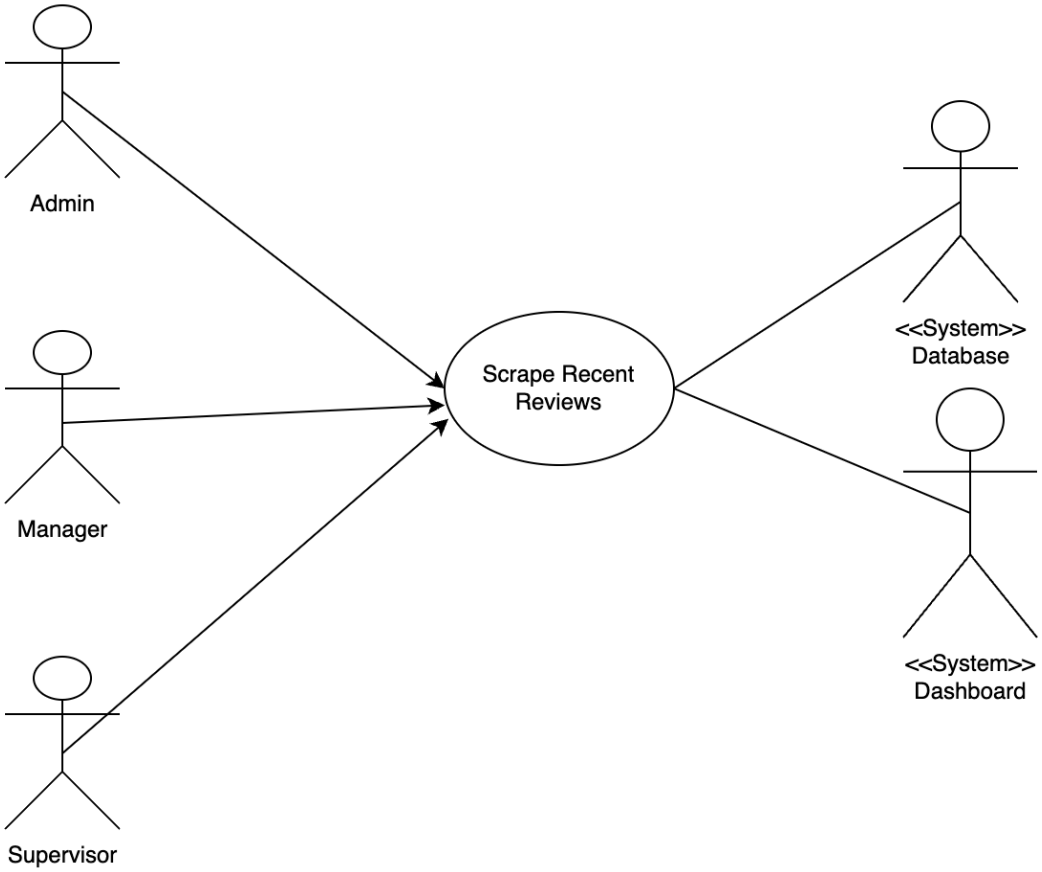
Preconditions: System has configured access to external sources

Main Flow:

1. User initiates the scraping process or system runs on schedule
2. System connects to external source (e.g., Google Reviews via Apify)
3. System collects recent reviews and filters by date range
4. System processes and stores the collected reviews
5. System displays a summary of collected data

Alternative Flows:

- Connection failure: System logs error and notifies admin
- No new reviews: System reports that no new data was found
- Postconditions: External review data is stored in the database and ready for analysis


Admin
Manager
Supervisor
Scrape Recent Reviews
<<System>> Database
<<System>> Dashboard

Recommendations

Goal: Analyze feedback trends over time to identify patterns

Actors: Admin, Manager, Supervisor

Preconditions: Historical feedback data exists in the system

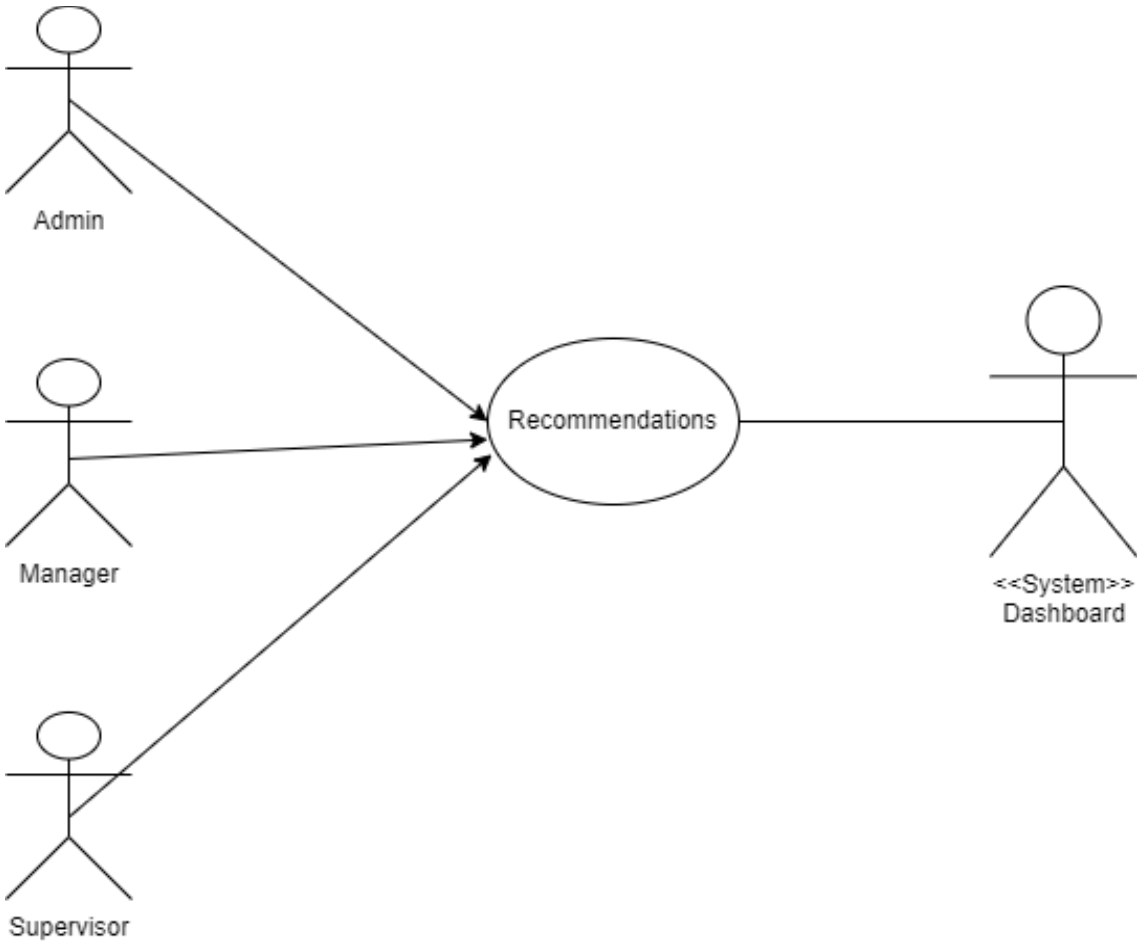
Main Flow:

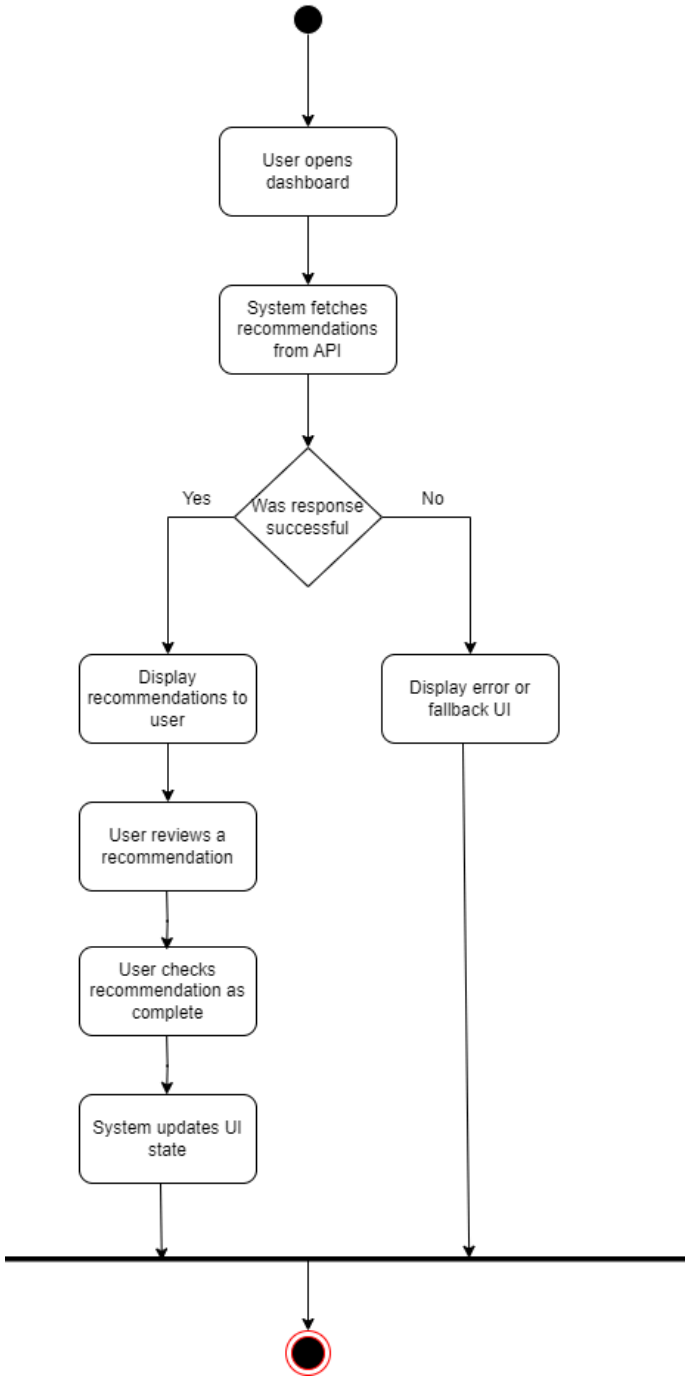
1. User navigates to the historical analysis section
2. User selects time period, departments, and metrics
3. System retrieves and processes historical data
4. System displays interactive visualizations of trends
5. User explores data through filtering and drill-down
6. System provides recommendations based on identified patterns
7. User views recommendations on the System Dashboard

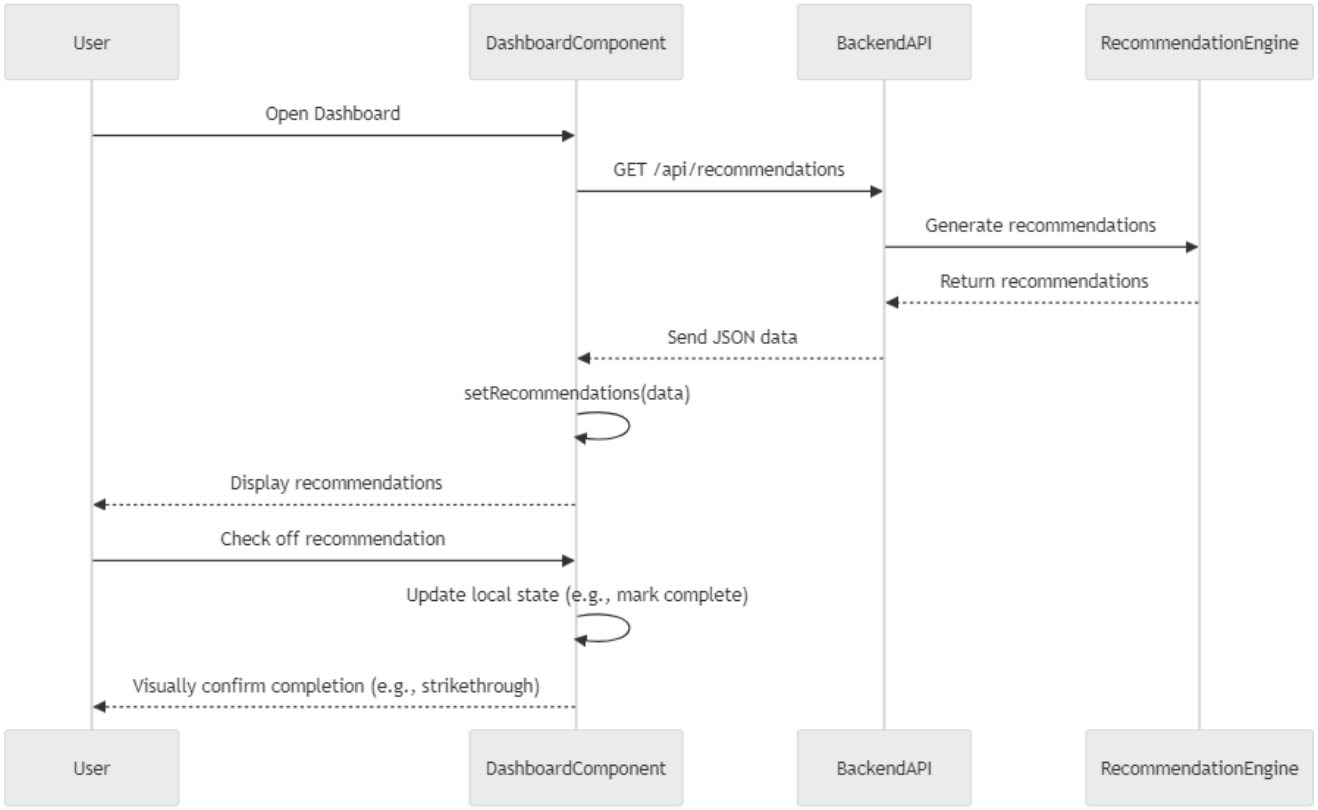
Alternative Flows:

- No historical data: System suggests collecting more data
- Complex query timeout: System suggests narrowing the analysis scope

Postconditions: User gains insights into historical trends and patterns; recommendations are displayed on the System Dashboard


Admin
Manager
Supervisor
Recommendations
<<System>>
Dashboard


User opens dashboard
System fetches recommendations from API
Was response successful
Yes
No
Display recommendations to user
Display error or fallback UI
User reviews a recommendation
User checks recommendation as complete
System updates UI state


User
DashboardComponent
BackendAPI
RecommendationEngine
Open Dashboard
GET /api/recommendations
Generate recommendations
Return recommendations
Send JSON data
setRecommendations(data)
Display recommendations
Check off recommendation
Update local state (e.g., mark complete)
Visually confirm completion (e.g., strikethrough)
User
DashboardComponent
BackendAPI
RecommendationEngine

Historical Analysis

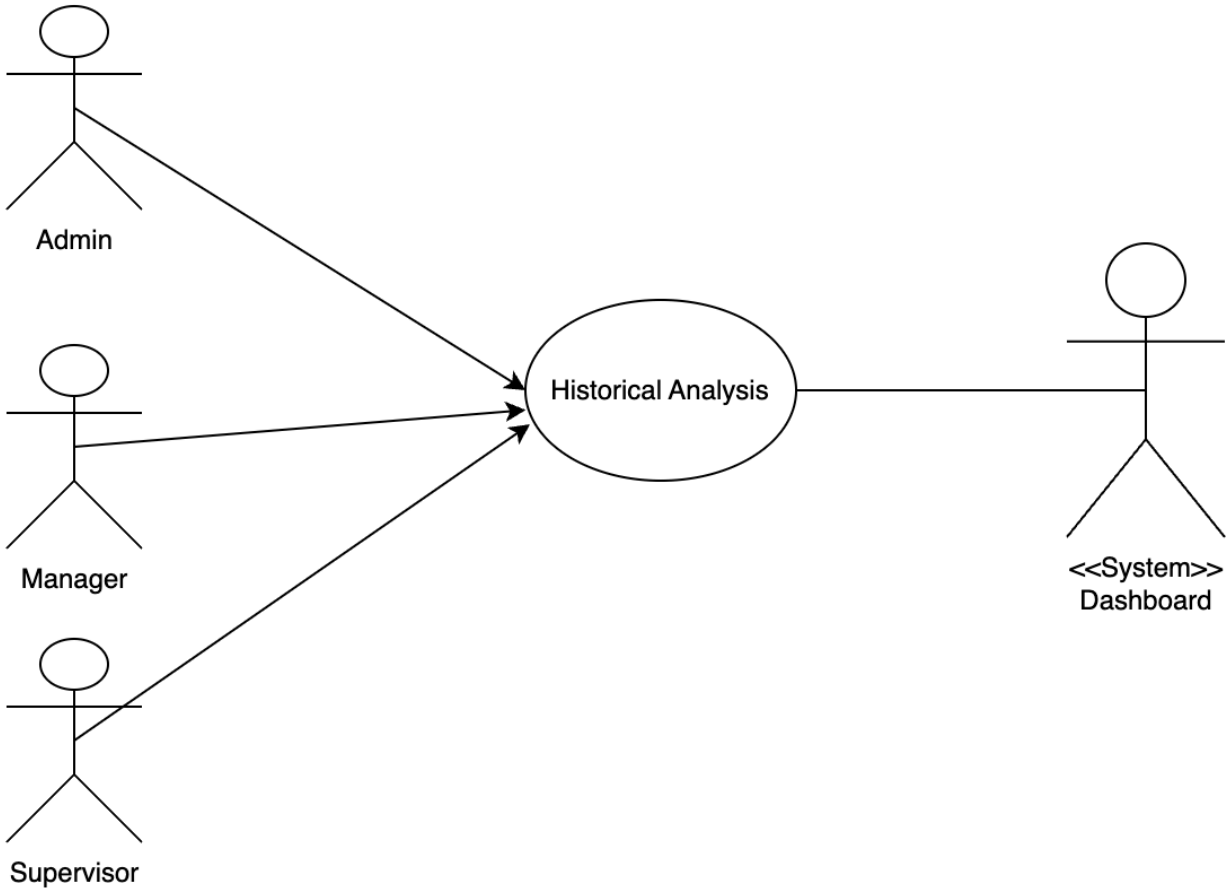
Goal: Analyze feedback trends over time to identify patterns
Actors: Admin, Manager, Supervisor
Preconditions: Historical feedback data exists in the system

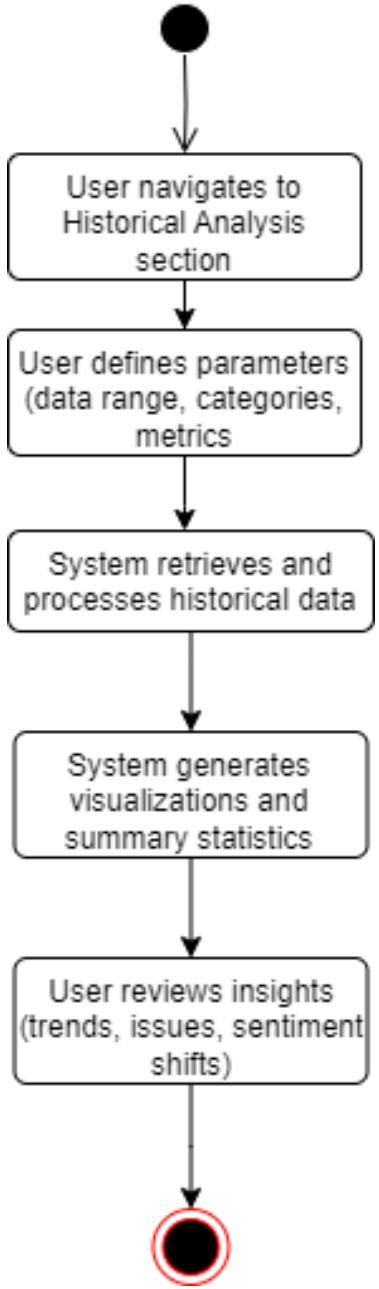
Main Flow:

1. User navigates to the historical analysis section
2. User selects time period, departments, and metrics
3. System retrieves and processes historical data
4. System displays interactive visualizations of trends
5. User explores data through filtering and drill-down

Alternative Flows:

- No historical data: System suggests collecting more data
- Complex query timeout: System suggests narrowing the analysis scope
- Postconditions: User gains insights into historical trends and patterns


Admin
Manager
Supervisor
Historical Analysis
<<System>>
Dashboard


User navigates to Historical Analysis section
User defines parameters (data range, categories, metrics
System retrieves and processes historical data
System generates visualizations and summary statistics
User reviews insights (trends, issues, sentiment shifts)


User
SystemInterface
Database
openHistoricalAnalysis()
displayAnalysisOptions()
setAnalysisParameters(dateRange, departments)
retrieveHistoricalData(dateRange, departments)
returnHistoricalData()
generateVisualizationsAndStats()
reviewInsights()
User
SystemInterface
Database

Create Alerts

Goal: Configure automatic notifications for specific feedback conditions

Actors: Admin, Manager

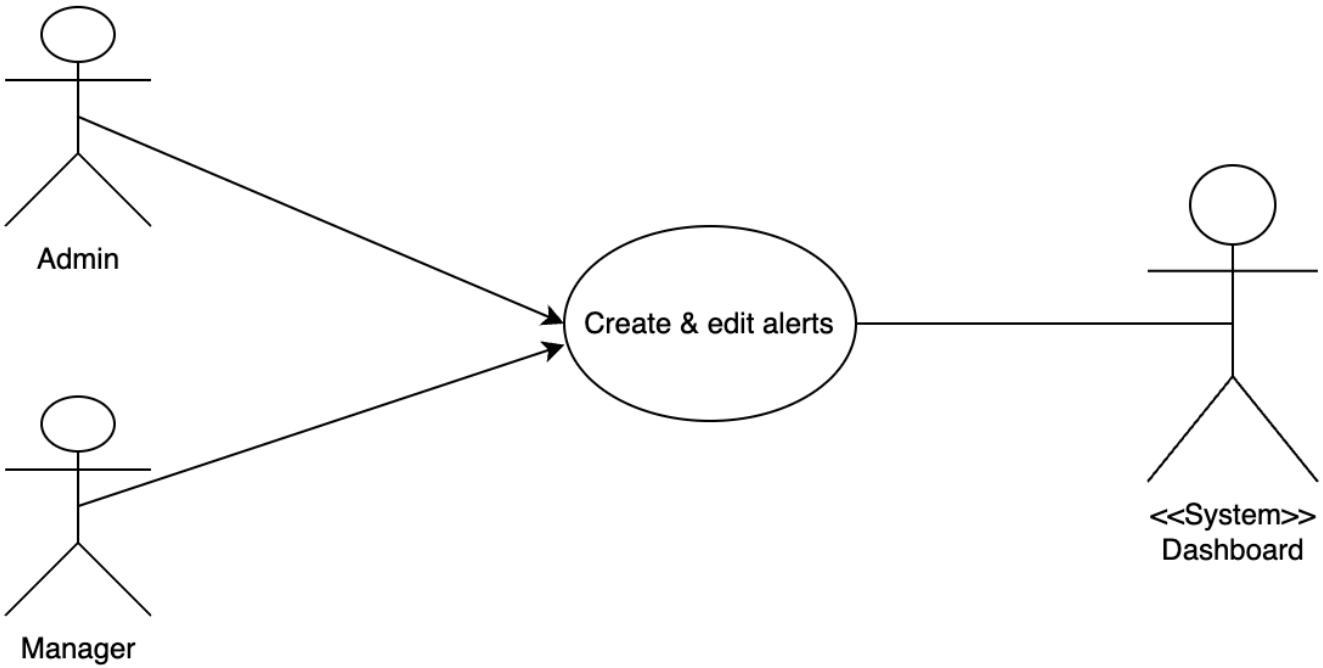
Preconditions: User has appropriate permissions

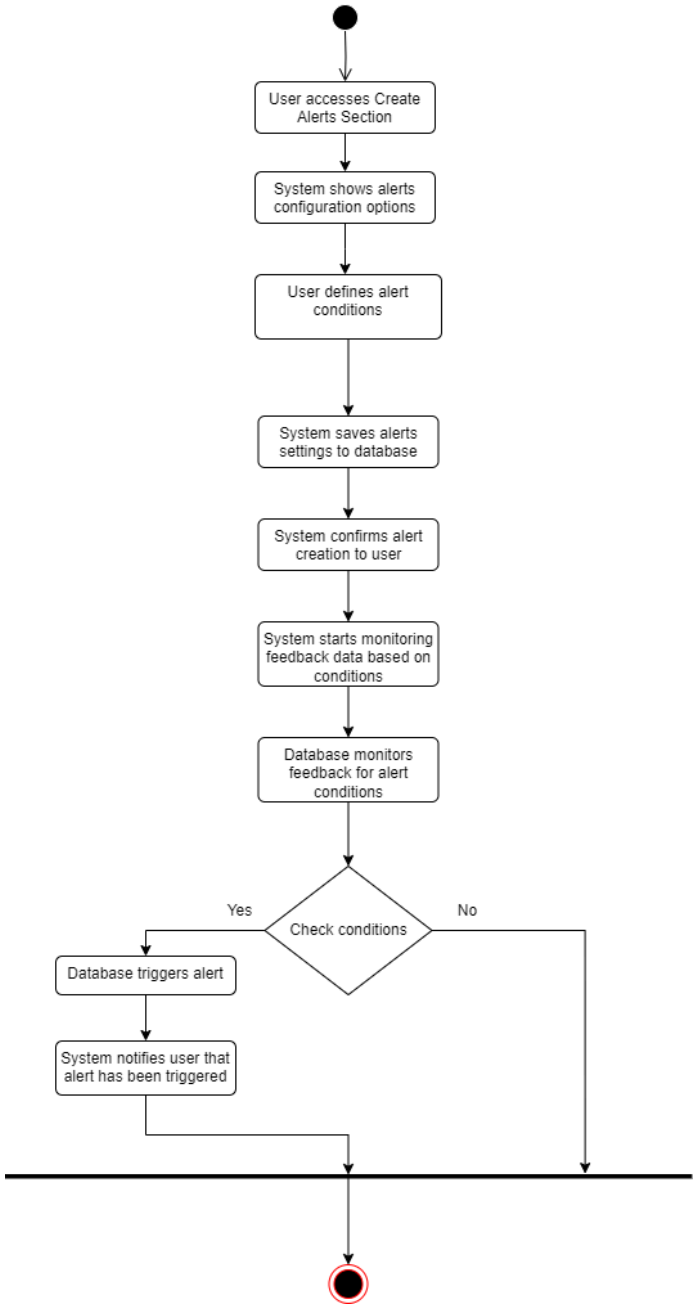
Main Flow:

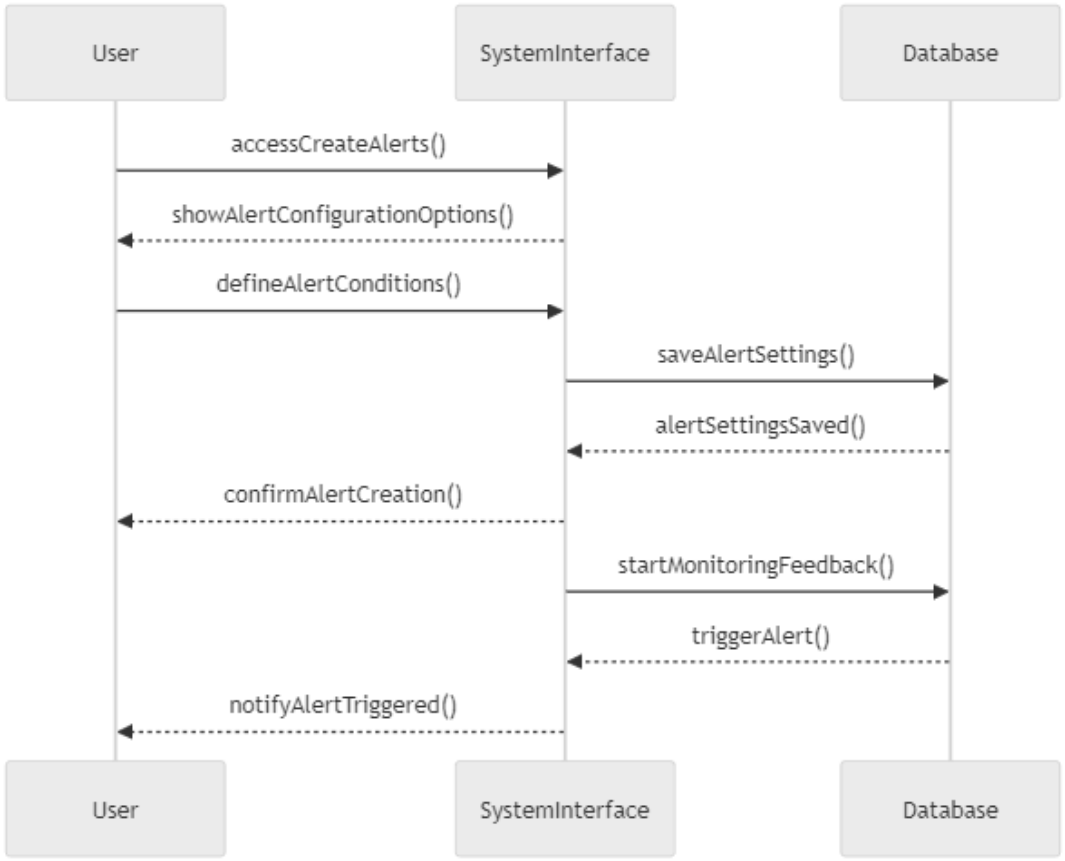
1. User navigates to the alerts management section
2. User defines alert criteria (department, sentiment threshold, etc.)
3. User sets alert priority and notification preferences
4. System saves the alert configuration
5. System confirms successful creation

Alternative Flows:

- Invalid criteria: System guides user to correct configuration
- Duplicate alert: System warns about similar existing alerts
- Postconditions: Alert is configured and active in the system


Admin
Manager
Create & edit alerts
<<System>>
Dashboard


User accesses Create Alerts Section
System shows alerts configuration options
User defines alert conditions
System saves alerts settings to database
System confirms alert creation to user
System starts monitoring feedback data based on conditions
Database monitors feedback for alert conditions
Check conditions
Yes
No
Database triggers alert
System notifies user that alert has been triggered


User
SystemInterface
Database
accessCreateAlerts()
showAlertConfigurationOptions()
defineAlertConditions()
saveAlertSettings()
alertSettingsSaved()
confirmAlertCreation()
startMonitoringFeedback()
triggerAlert()
notifyAlertTriggered()
User
SystemInterface
Database

Receive Alerts

Goal: Notify users when alert conditions are met

Actors: Admin, Manager, Supervisor

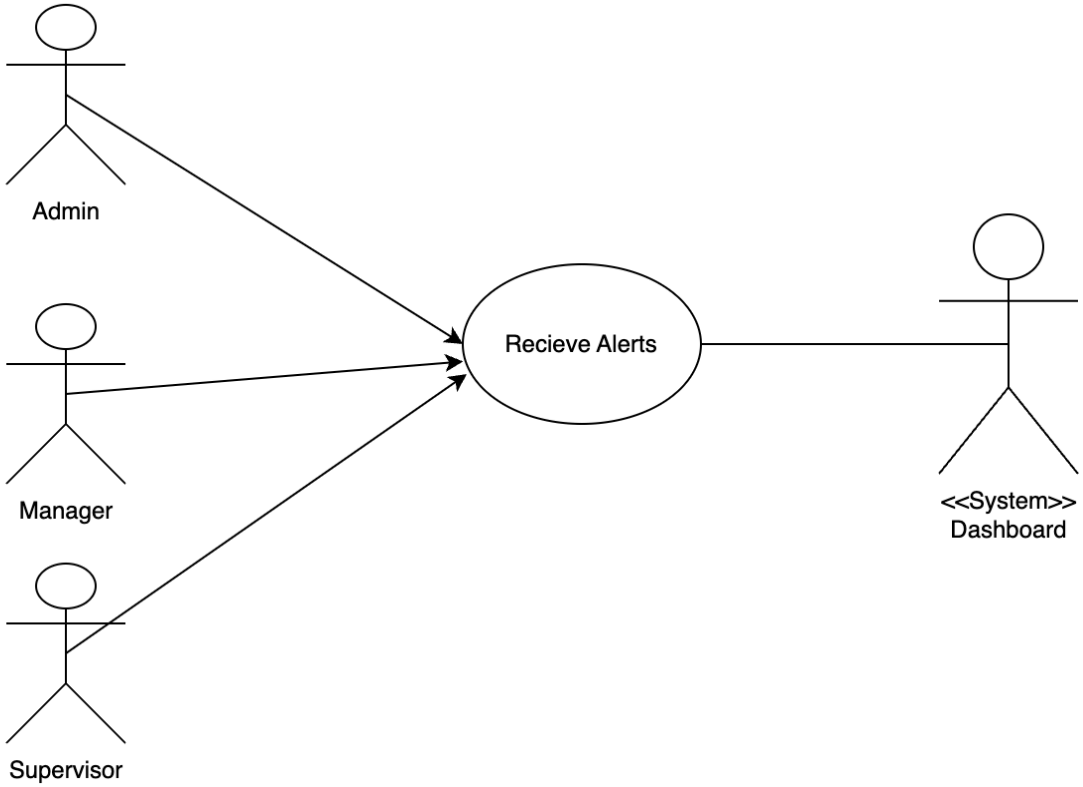
Preconditions: Alert configuration exists and new feedback is processed

Main Flow:

1. System processes new feedback
2. System checks if feedback matches any alert criteria
3. When criteria are met, system generates an alert
4. System notifies relevant users based on department and role
5. Users view and acknowledge alerts through the dashboard

Alternative Flows:

- No matching criteria: No alerts are generated
- System failure: Alerts are queued for later delivery
- Postconditions: Users are informed about important feedback that requires attention


Admin
Manager
Supervisor
Recieve Alerts
<<System>>
Dashboard

Results

Goal: Allow Admins, Managers, and Supervisors to view and analyze results within the system. The results can also generate reports for further analysis.

Actors: Admin, Manager, Supervisor

Preconditions:

- User is authenticated and authorized to view results.
- Results data is available in the system.

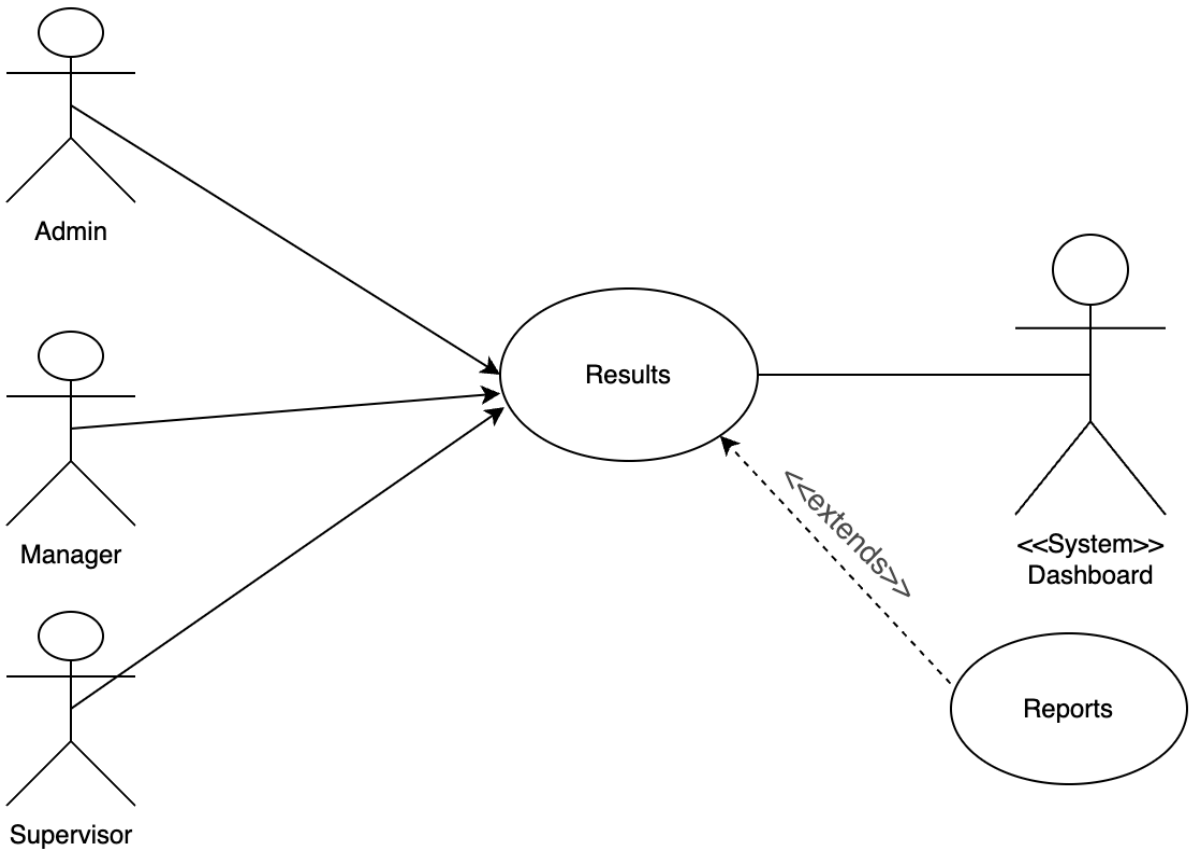
Main Flow:

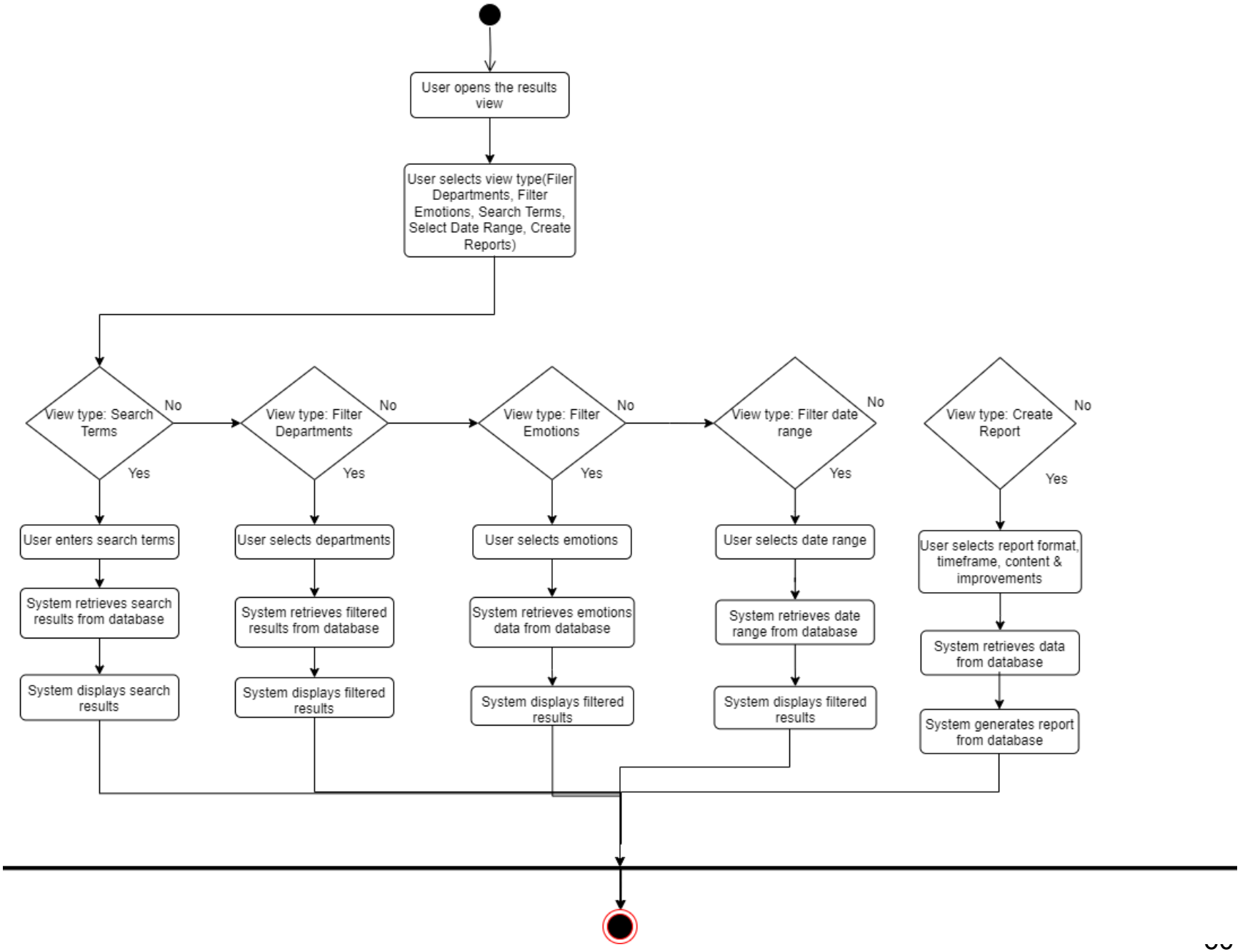
1. User navigates to the results section.
2. The system retrieves the relevant results from the database.
3. The system displays results using graphical representations, charts, or tables.
4. Users can filter or sort results based on different criteria.
5. Users can select the option to generate a report if needed.
6. System generates a report and displays a confirmation.

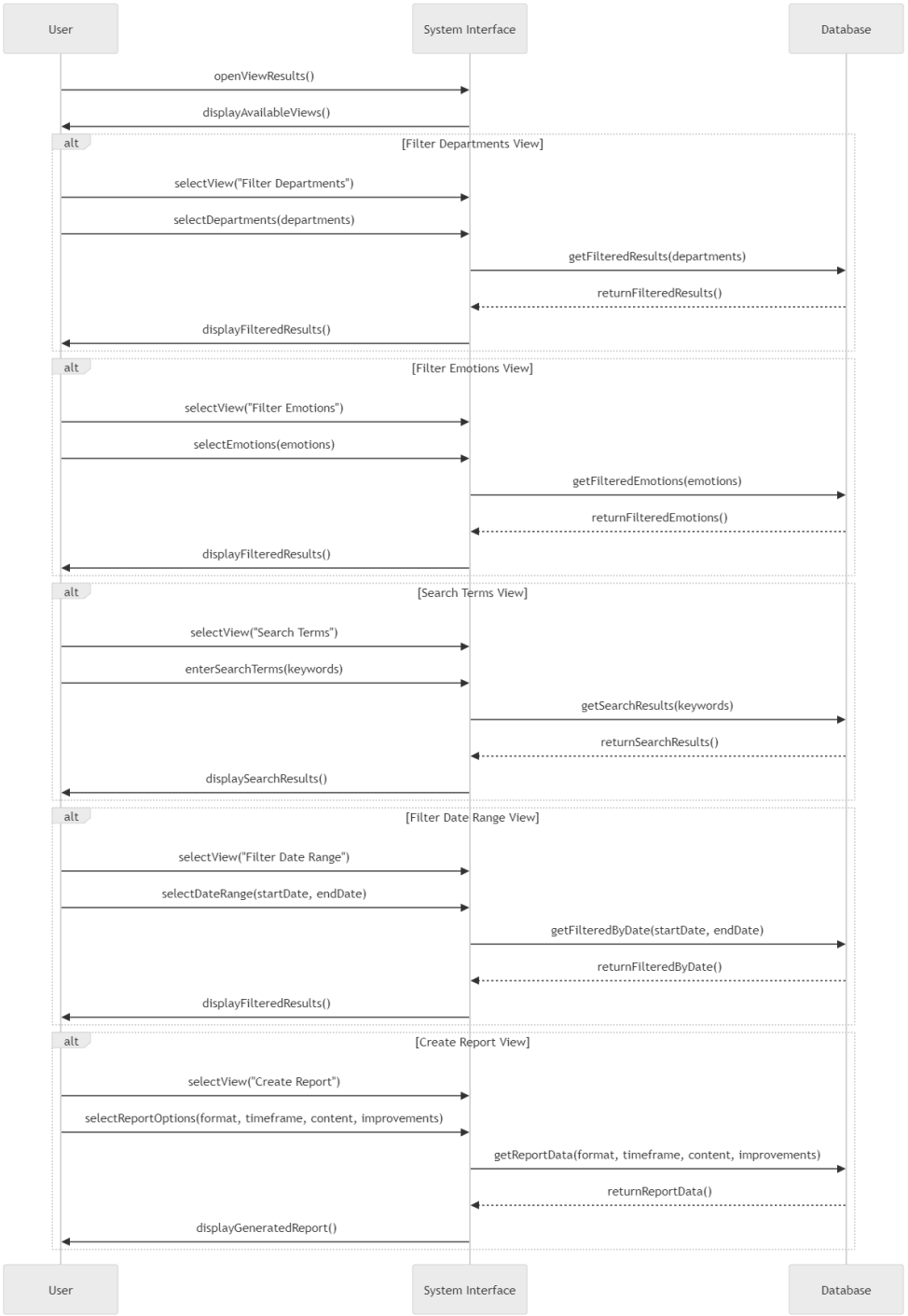
Alternative Flows:

- Data Unavailable: If no results are available for the selected criteria, the system displays a notification indicating no data is available.
- Report Generation Failed: If report generation fails due to system issues, the system displays an error message and logs the error for troubleshooting.
- Permission Denied: If the user lacks permission to access specific result data, the system denies access and displays a relevant error message.
- **Postconditions:**

- The user views the required results.
- If applicable, a report is generated and stored within the system for further use.


Admin
Manager
Supervisor
Results
<<extends>>
<<System>>
Dashboard
Reports


User opens the results view
User selects view type(Filer Departments, Filter Emotions, Search Terms, Select Date Range, Create Reports)
View type: Search Terms
No
Yes
User enters search terms
System retrieves search results from database
System displays search results
View type: Filter Departments
No
Yes
User selects departments
System retrieves filtered results from database
System displays filtered results
View type: Filter Emotions
No
Yes
User selects emotions
System retrieves emotions data from database
System displays filtered results
View type: Filter date range
No
Yes
User selects date range
System retrieves date range from database
System displays filtered results
View type: Create Report
No
Yes
User selects report format, timeframe, content & improvements
System retrieves data from database
System generates report from database


User
System Interface
Database
openViewResults()
displayAvailableViews()
alt
[Filter Departments View]
selectView("Filter Departments")
selectDepartments(departments)
getFilteredResults(departments)
returnFilteredResults()
displayFilteredResults()
alt
[Filter Emotions View]
selectView("Filter Emotions")
selectEmotions(emotions)
getFilteredEmotions(emotions)
returnFilteredEmotions()
displayFilteredResults()
alt
[Search Terms View]
selectView("Search Terms")
enterSearchTerms(keywords)
getSearchResults(keywords)
returnSearchResults()
displaySearchResults()
alt
[Filter Date Range View]
selectView("Filter Date Range")
selectDateRange(startDate, endDate)
getFilteredByDate(startDate, endDate)
returnFilteredByDate()
displayFilteredResults()
alt
[Create Report View]
selectView("Create Report")
selectReportOptions(format, timeframe, content, improvements)
getReportData(format, timeframe, content, improvements)
returnReportData()
displayGeneratedReport()
User
System Interface
Database

Logout

Goal: Allow users to securely end their session

Actors: Admin, Manager, Supervisor

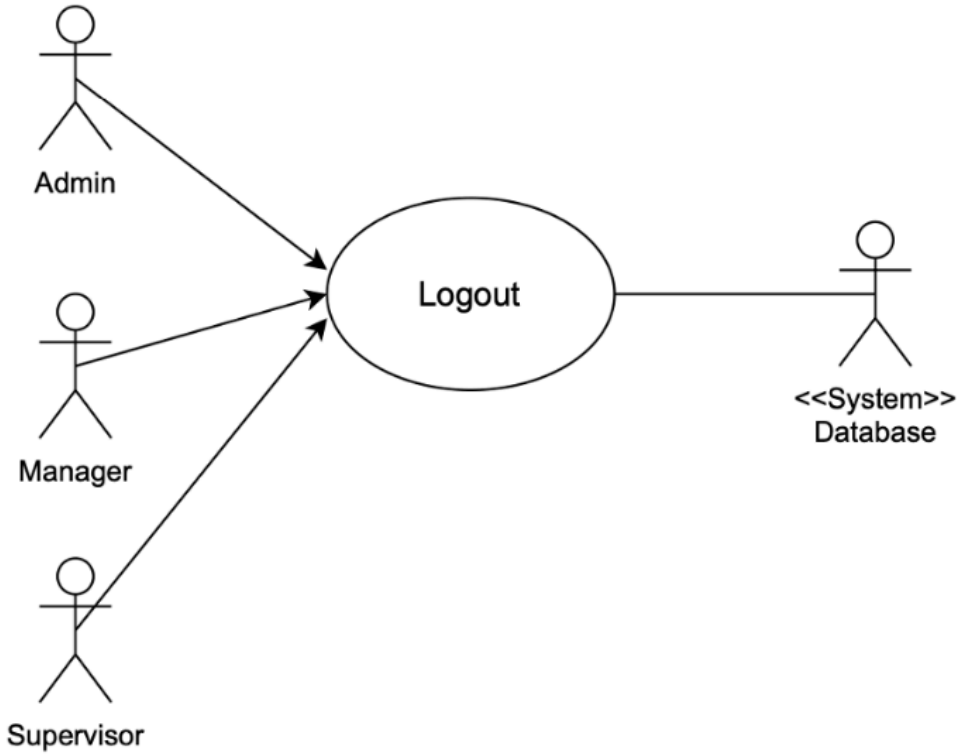
Preconditions: User is currently logged in

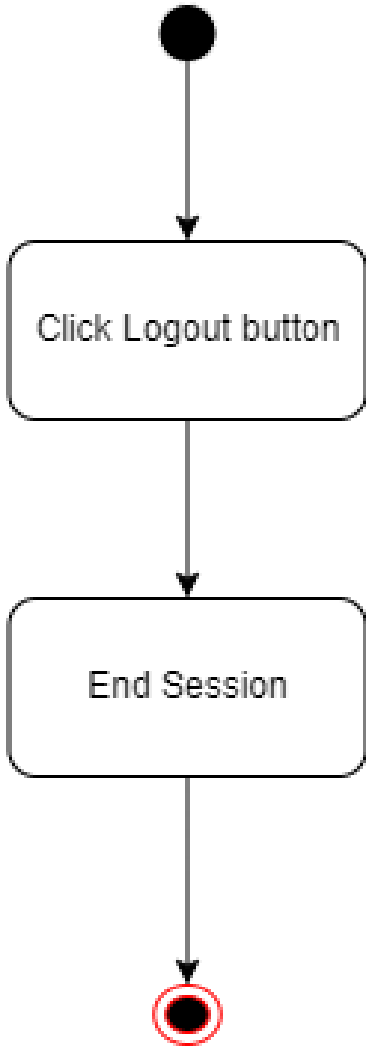
Main Flow:

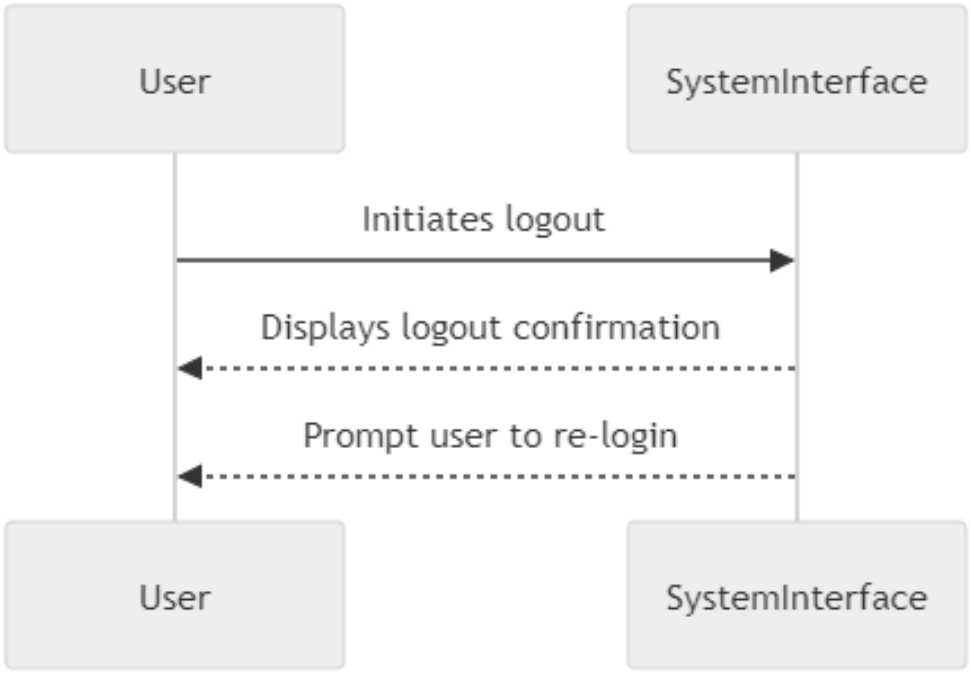
1. User clicks on the logout option in the navigation menu
2. System terminates the user's session
3. System redirects to the login page

Alternative Flows:

- Postconditions: User session is terminated and access is revoked


Admin
Manager
Supervisor
Logout
<<System>>
Database


Click Logout button
End Session


User
SystemInterface
Initiates logout
Displays logout confirmation
Prompt user to re-login
User
SystemInterface

Feedback Processing

- Sentiment Analysis:
    - Goal: Analyze feedback text to determine sentiment and emotion
    - Actors: System
    - Preconditions: Feedback data exists in the system
    - Main Flow:
        1. System retrieves unprocessed feedback from the database
        2. System sends text to the NLP service for analysis
        3. NLP service determines sentiment (positive, negative, neutral) and confidence score
        4. NLP service identifies emotions present in the text
        5. System stores analysis results with the original feedback
    - Alternative Flows:
        1. NLP service unavailable: System queues feedback for later processing
        2. Analysis error: System flags feedback for manual review
    - Postconditions: Feedback is enriched with sentiment and emotion data
- Department Classification:
    - Goal: Assign feedback to relevant hotel departments
    - Actors: System
    - Preconditions: Feedback data with sentiment analysis exists
    - Main Flow:
        1. System analyzes feedback text for department-specific keywords
        2. System applies classification rules to determine the most relevant department
        3. System assigns departmental tags to the feedback
        4. System updates the feedback record with department information
    - Alternative Flows:
        1. Ambiguous department: System assigns to "General" category

2. Multiple departments: System assigns primary and secondary departments
    - Postconditions: Feedback is organized by department for targeted analysis

Component Design

User Management

The User Management component handles authentication, authorization, and user profile management:

Key Classes/Components:

- User model: Stores user information, credentials, role and department
- AuthController: Handles login, logout, and session management
- UserController: Manages user creation, updates, and role assignments
- PermissionService: Enforces role-based access control

Functionality:

- User authentication with secure password handling
- Role-based access control (Admin, Manager, Supervisor)
- Department-specific permissions for managers and supervisors
- User profile management
- Password reset capabilities
- Session management with JWT tokens

Interfaces:

- REST API endpoints for user operations
- Login/logout UI components
- User management dashboard for administrators

Upload Datasets

The Upload Datasets component manages the import of feedback data from files:

Key Classes/Components:

- UploadController: Handles file reception and initial validation
- FeedbackParser: Processes CSV and JSON files into standardized format
- ValidationService: Ensures data quality and required fields
- DuplicationService: Identifies and handles duplicate feedback entries

Functionality:

- File upload interface with drag-and-drop support
- Format validation for CSV and JSON files
- Data transformation into standardized internal format
- Error handling for malformed data
- Progress tracking for large uploads
- Duplicate detection and handling

Interfaces:

- File upload UI with progress indication
- Validation error reporting
- Success confirmation with summary statistics

Historical Analysis

The Historical Analysis component enables trend analysis over time periods:

Key Classes/Components:

- AnalyticsService: Processes historical data for trends
- TimeSeriesAnalyzer: Identifies patterns in time-based data
- ComparisonEngine: Compares different time periods
- VisualizationService: Prepares data for charting

Functionality:

- Selection of time periods for analysis
- Department and category filtering
- Trend identification and highlighting
- Comparative analysis between periods
- Interactive visualizations
- Insight generation based on statistical analysis

Interfaces:

- Time period selection controls
- Department/category filters
- Interactive charts and graphs
- Trend indicators and highlights
- Export options for analysis results

Results Viewing

The Results component displays analyzed feedback data with filtering and search capabilities:

Key Classes/Components:

- ResultsController: Manages data retrieval and filtering
- SearchService: Handles keyword and criteria-based searching
- FilterEngine: Applies user-selected filters to results
- SortingService: Orders results based on various criteria

Functionality:

- Comprehensive dashboard of feedback metrics
- Multi-criteria filtering (department, sentiment, date, etc.)
- Keyword search across feedback text
- Sorting and prioritization of results

- Detailed view of individual feedback items
- Export capabilities for filtered results

Interfaces:

- Dashboard with key metrics
- Advanced filtering controls
- Search functionality
- Results display with sorting options
- Detailed feedback viewer

Recommendation Engine

The Recommendation Engine identifies recurring issues and suggests improvements:

Key Classes/Components:

- Recommendation Service: Generates action suggestions from feedback
- Pattern Detector: Identifies recurring negative keywords in negative feedback
- Rule Engine: Applies predefined rules to match issues with solutions
- Priority Calculator: Determines importance of recommendations

Functionality:

- Analysis of negative feedback for patterns
- Matching of identified negative keywords to predefined solutions
- Prioritization based on frequency and impact
- Tracking of implemented recommendations

Interfaces:

- Recommendation dashboard
- Priority indicators
- Implementation tracking

Data Dictionary and Schema

Database Models

FeedbackFusion uses MongoDB as its primary database, with the following core models:

User Model:

Copy

```
{
  "userId": "ObjectId", // Unique identifier
  "username": "String", // User's login name
  "password": "String", // Hashed password
  "role": "String", // 'admin', 'manager', or 'supervisor'
  "assignedDepartments": ["ObjectId"], // References to Department model
  "lastLogin": "Date" // Timestamp of last login
}
```

Department Model:

Copy

```
{
  "departmentId": "ObjectId", // Unique identifier
  "name": "String", // Department name (e.g., "Housekeeping")
  "categories": ["String"] // Categories within the department
}
```

Feedback Model:

Copy

```
{
  "feedbackId": "ObjectId", // Unique identifier
  "feedback_text": "String", // Original feedback text
  "textTranslated": "String", // Translated text (if applicable)
```

```
  "rating": "Number", // Numerical rating (e.g., 1-5)
  "date": "Date", // Date of feedback
  "source": "String", // Source of feedback (e.g., "Google Reviews")
  "department": "String", // Assigned department
  // NLP Analysis results
  "sentiment_label": "String", // Sentiment classification
  "sentiment_confidence": "Number", // Confidence score
  "emotion_label": "String", // Identified emotion
  "emotion_confidence": "Number", // Confidence score
  // Recommendations based on feedback
  "recommendations": [
    {
      "keyword": "String", // Issue keyword
      "solutions": ["String"] // Suggested solutions
    }
  ]
}
```

Alert Model:

Copy

```
{
  "alertId": "ObjectId", // Unique identifier
  "department": "String", // Department this alert applies to
  "priority": "String", // 'low', 'medium', or 'high'
  "threshold": "Number", // Threshold to trigger alert
  "active": "Boolean", // Whether alert is active
  "triggered": "Boolean", // Whether alert has been triggered
  "triggeredAt": "Date", // When alert was triggered
  "createdAt": "Date" // When alert was created
}
```

Dataset Model:

Copy

```
{
  "datasetId": "ObjectId", // Unique identifier
  "name": "String", // Dataset name
  "fileType": "String", // 'CSV' or 'JSON'
  "uploadedOn": "Date", // Upload timestamp
  "uploadedBy": "ObjectId", // Reference to User model
  "feedbackIds": ["ObjectId"] // References to created Feedback entries
}
```

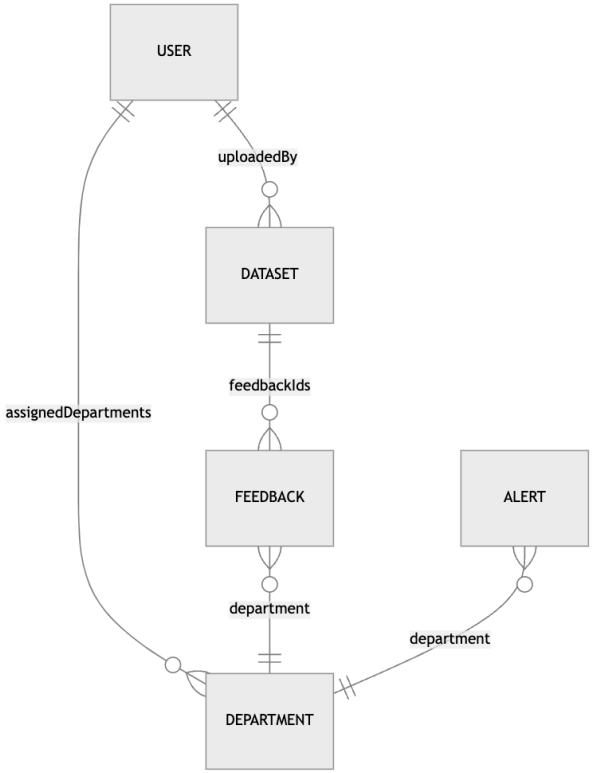
Historical Analysis Model:

Copy

```
{
  "analysisId": "ObjectId", // Unique identifier
  "parameters": "Object", // Analysis parameters
  "dateRange": { // Time period for analysis
    "start": "Date",
    "end": "Date"
  },
  "insights": ["String"] // Generated insights
}
```

Schema Relationships

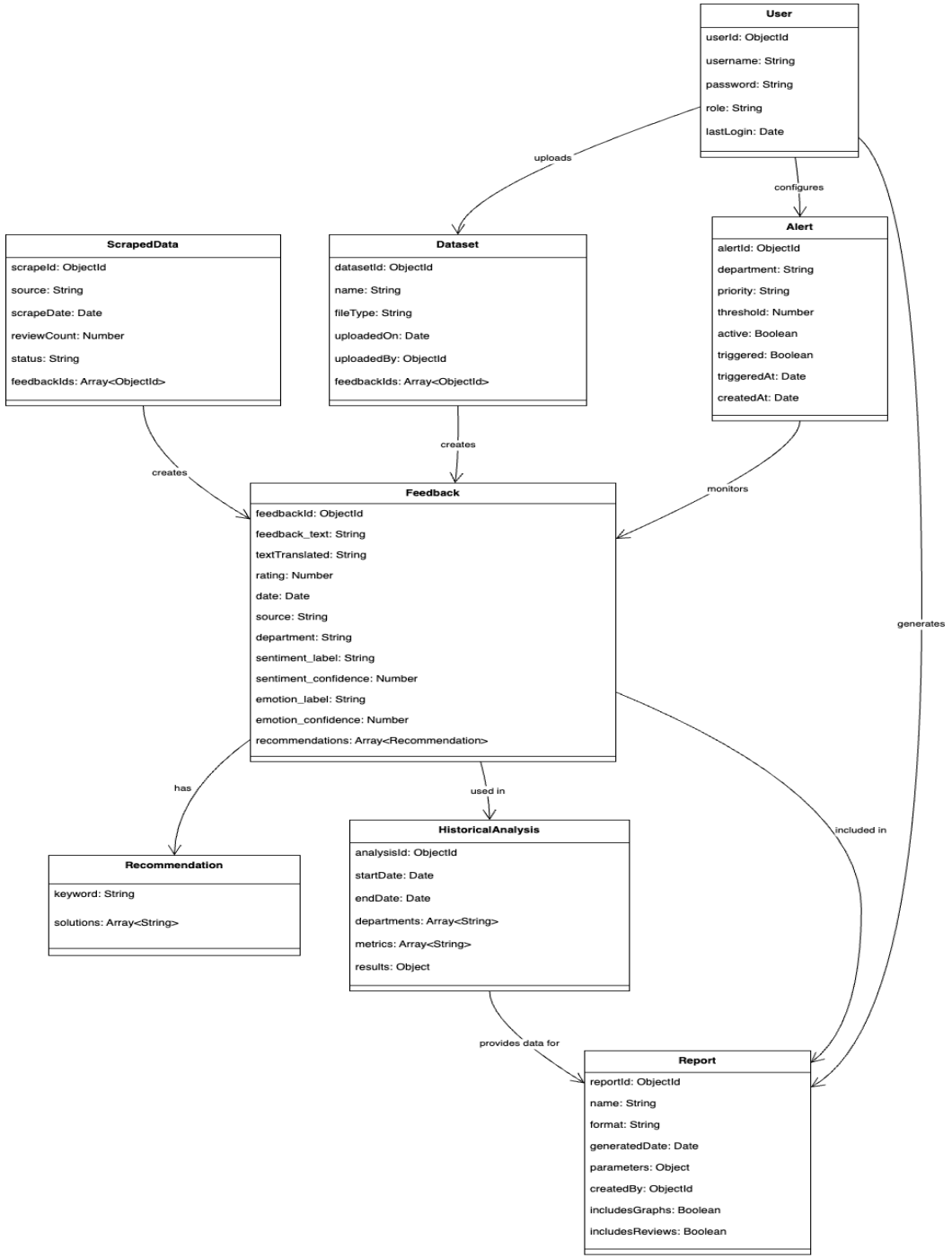
The relationships between the data models are as follows:



1. User to Department:
    - Many-to-many: Users (particularly managers and supervisors) can be assigned to multiple departments
    - Implemented through the assignedDepartments array in the User model
1. Feedback to Department:
    - Many-to-one: Each feedback item is assigned to one primary department
    - Implemented through the department field in the Feedback model
1. Dataset to Feedback:

    - One-to-many: A dataset can create multiple feedback entries
    - Implemented through the feedbackIds array in the Dataset model
1. User to Dataset:
    - One-to-many: A user can upload multiple datasets
    - Implemented through the uploadedBy field in the Dataset model
1. Alert to Department:
    - Many-to-one: Multiple alerts can be configured for a single department
    - Implemented through the department field in the Alert model

Class Diagram

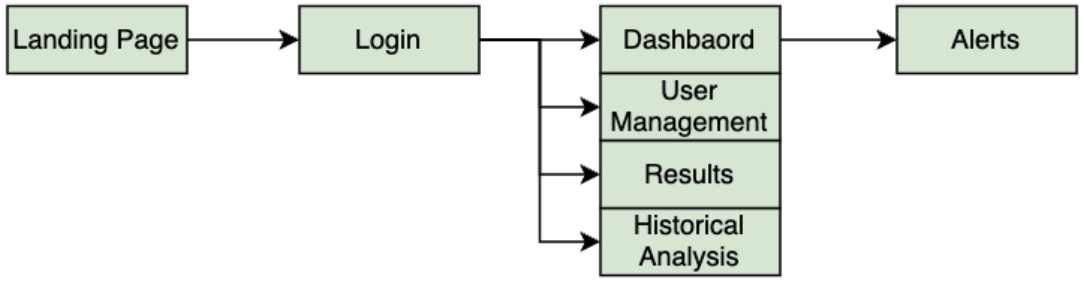


This class diagram represents the data model for a guest feedback management and analysis system. At its core is the Feedback class, which stores individual feedback entries, including sentiment and emotion labels, confidence scores, and linked Recommendation objects generated based on negative keywords. Feedback entries are created from either ScrapedData or manually uploaded Datasets, both of which maintain references to their generated feedback. The User class represents system users (e.g., admins, managers), who can upload datasets, configure Alerts for specific departments, and generate Reports. Alerts monitor feedback patterns and trigger based on thresholds. Feedback data can also be grouped into HistoricalAnalysis entities for trend insights over time, with results powering comprehensive Reports. The relationships capture a full data lifecycle—from data ingestion and NLP-based analysis to recommendation generation, monitoring, and reporting—providing a structured foundation for decision-making in hospitality environments.

User Interface Design

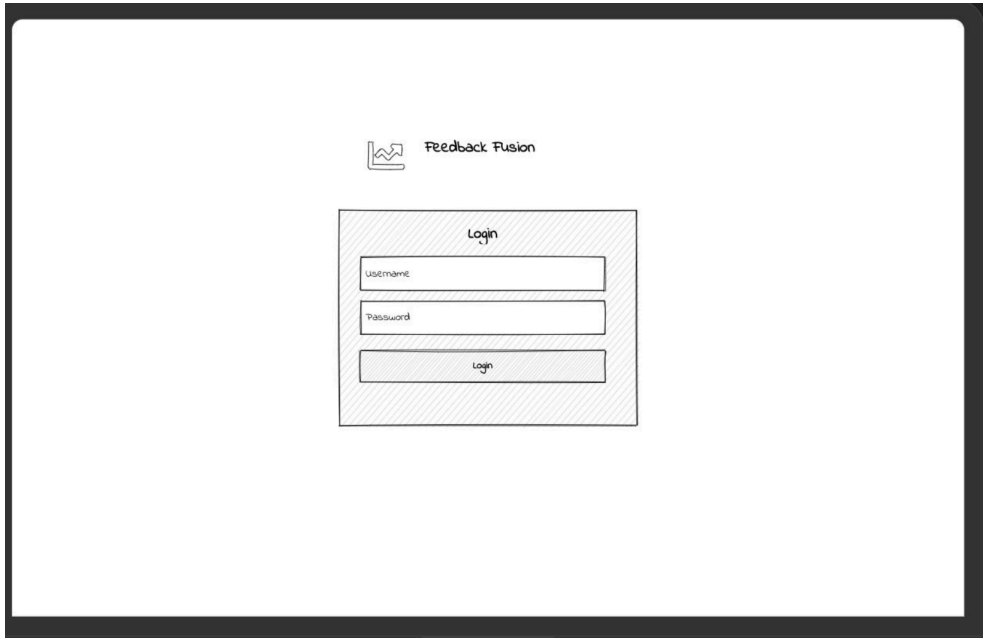
Overview

FeedbackFusion's user interface follows a clean, intuitive design that emphasizes data visualization and ease of navigation. The interface is organized around several key screens:


Landing Page
Login
Dashbaord
User Management
Results
Historical Analysis
Alerts

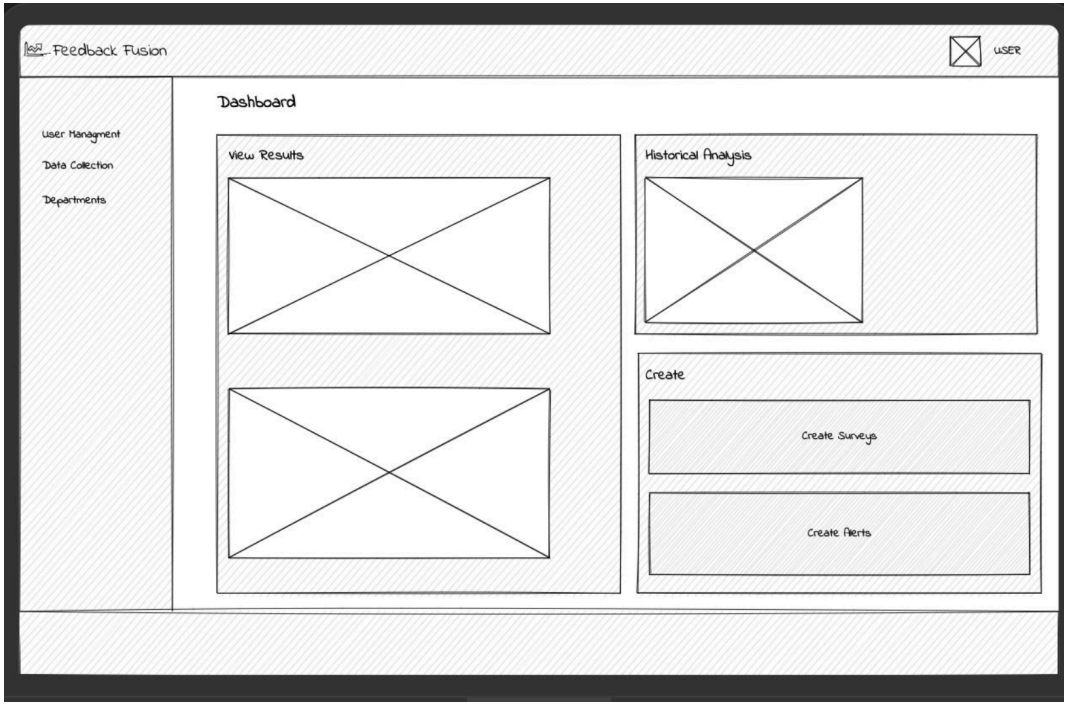
Login Screen:

- Minimalist design with logo and branding
- Username and password fields
- Login button
- Password reset option



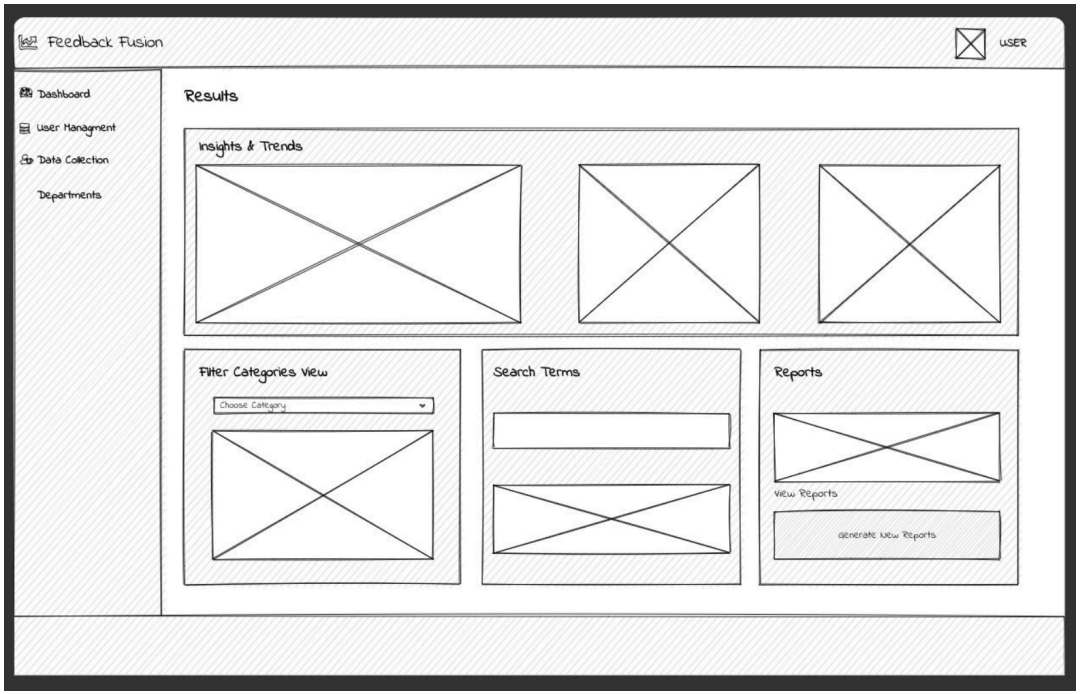
Dashboard Screen:

- Overview of key metrics (average rating, sentiment score, recent reviews, etc.)
- Recent alerts and notifications
- Recent negative keywords and recommendations
- Summary charts for sentiment trends and department performance



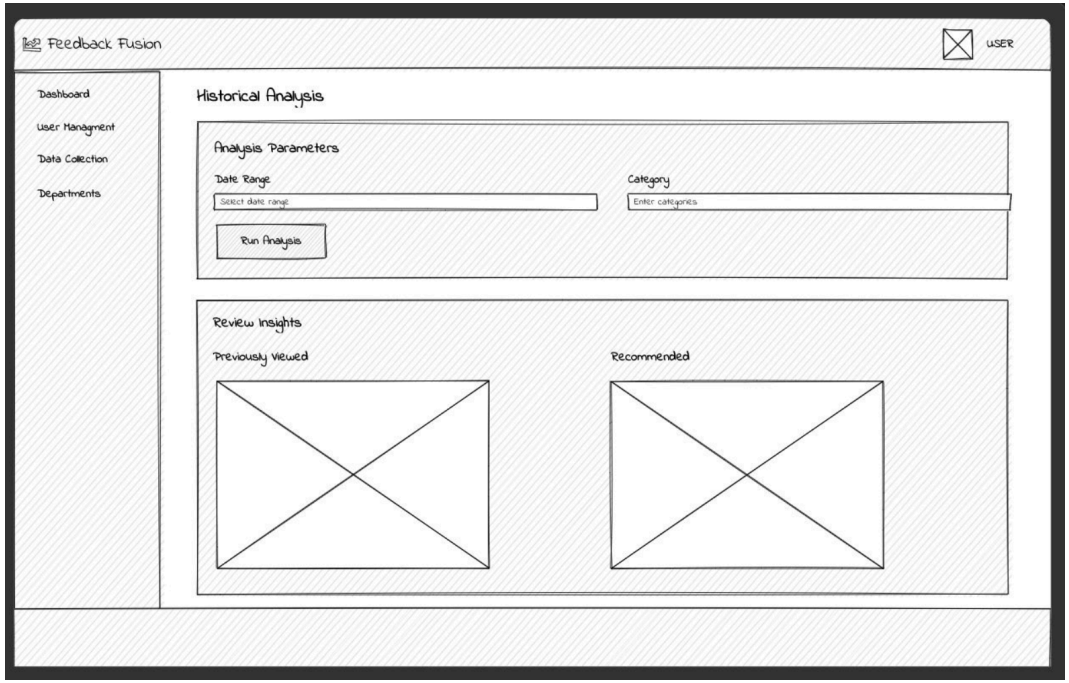
Results Screen:

- Comprehensive filtering options (date range, department, sentiment, emotion)
- Keyword search functionality
- Interactive charts showing sentiment distribution and trends
- Tabular display of feedback entries with sorting and pagination
- Detail view for individual feedback items



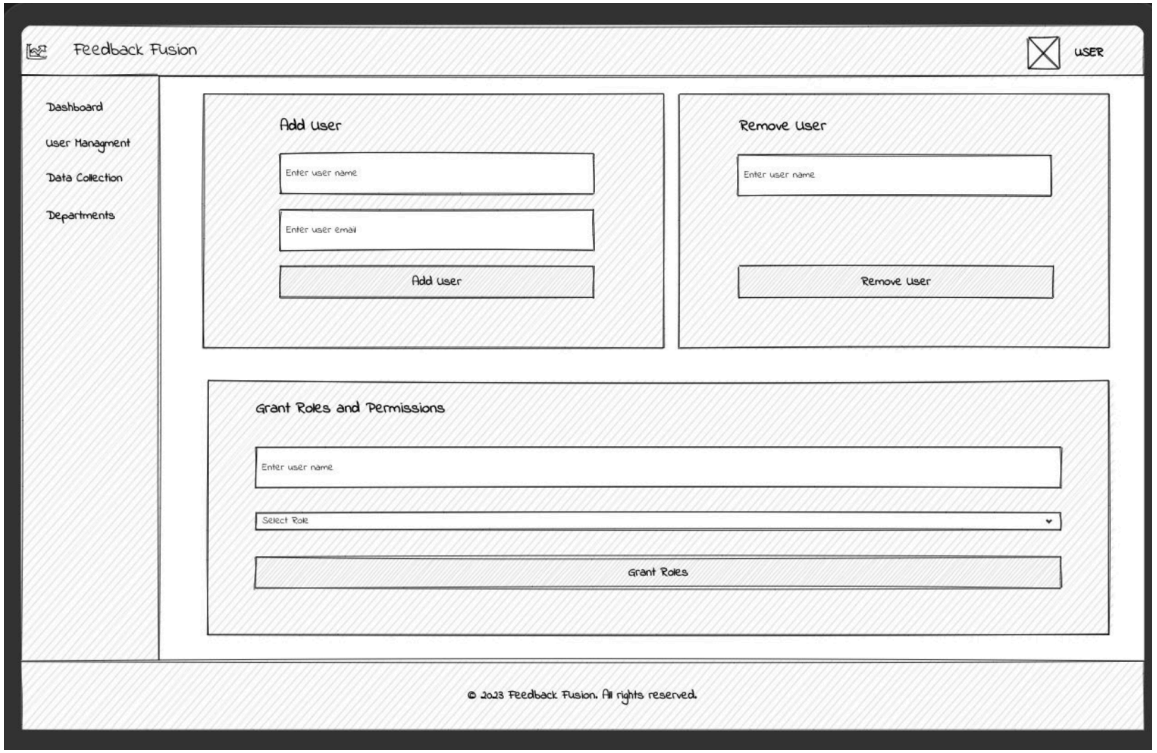
Historical Analysis Screen:

- Date range selection for trend analysis
- Department filter
- Multiple visualization types (sentiment trend, department comparison, review volume, emotion distribution, etc.)
- Comparative analysis tools



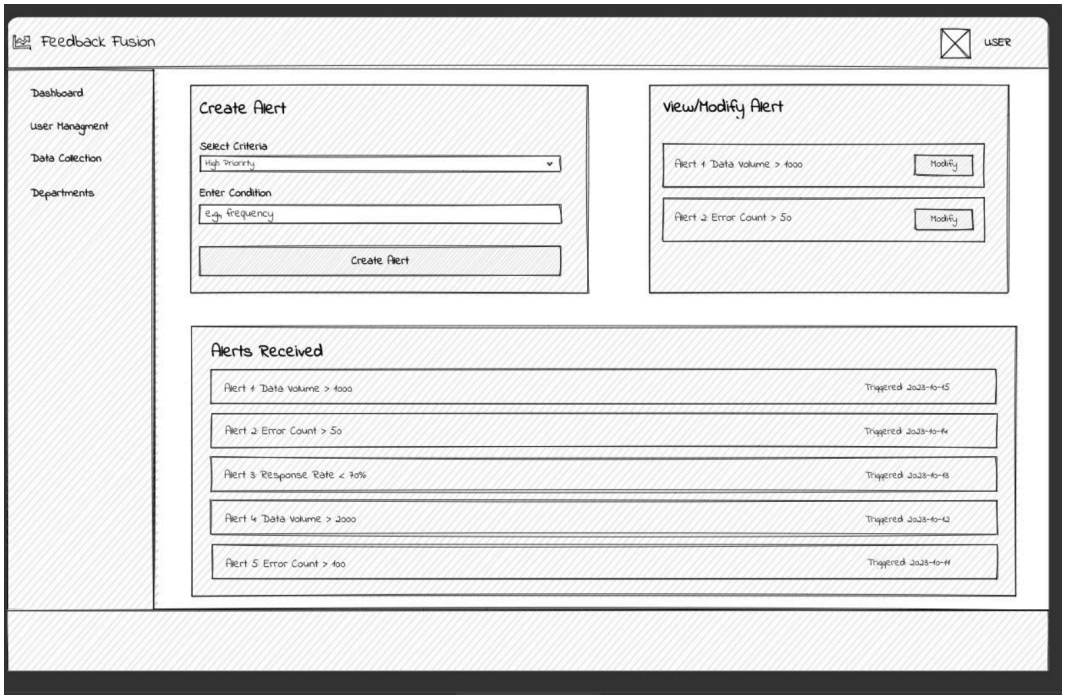
User Management Screen (Admin only):

- User listing with role information
- Add and delete user interface
- Edit user capabilities
- Role assignment controls



Alert Management Screen:

- Current alerts overview
- Alert creation interface
- Threshold configuration
- Department assignment
- Priority setting



Dashboard Design

The dashboard is the central hub of the FeedbackFusion system, designed to provide at-a-glance information and quick access to detailed features:

Layout Structure:

- Top Navigation Bar: Upload, scrape recent reviews & notifications buttons
- Side Navigation Panel: Access to: Results, HIstorical Analysis, User Management & Profile (logout & change password)
- Main Content Area: Dashboard widgets and visualizations
- Action Bar: Context-specific actions and filters

Key Dashboard Components:

1. Metric Cards: Display critical KPIs such as:
    - Average rating score
    - Sentiment distribution
    - Total review count
    - Recent trend indicator
1. Sentiment Trend Chart: Line chart showing sentiment trend over time, allowing users to identify patterns and changes in guest satisfaction.
2. Department Performance: Bar chart or radar chart displaying rating and sentiment scores by department, highlighting areas of strength and concern.
3. Recent Reviews Panel: Displays the latest feedback entries with sentiment indicators, allowing quick access to review details.
4. Alert Section: Shows active alerts that require attention, sorted by priority with visual indicators.
5. Recommendation Widget: Presents actionable suggestions based on feedback patterns, focusing on high-impact improvements.

Visual Design Elements:

- Color Scheme: Professional color palette with semantic use of colors:
    - Green tones for positive sentiment
    - Red tones for negative sentiment
    - Blue tones for neutral elements
    - Yellow/orange for alerts and warnings

- Typography: Clean, readable fonts with clear hierarchy:
    - Headings: Bold, slightly larger
    - Body text: Regular weight, optimized for readability
    - Data labels: Compact but legible
- Data Visualization: Consistent chart styles with:
    - Clear legends and labels
    - Appropriate chart types for different data
    - Interactive elements (hover tooltips, click-through for details)
    - Responsive sizing for different screen dimensions

Navigation Structure

FeedbackFusion employs a hierarchical navigation structure that provides both context awareness and efficient access to features:

Primary Navigation:

- Dashboard: System overview and key metrics
- Results: Detailed feedback exploration and filtering
- Historical Analysis: Trend analysis over time
- Alerts: Alert management and configuration
- User Management (Admin only): User administration tools

Secondary Navigation:

- Context-specific sub-navigation appears within primary sections
- Breadcrumb navigation shows current location in the system
- Recent locations remembered for quick navigation back

Navigation Patterns:

- Sidebar Navigation: Persistent side menu for main feature access

- Tab Navigation: Used within features for related sub-views
- Card Navigation: Dashboard cards can be clicked for detailed views
- Filter-Based Navigation: Results view allows navigation through data via filtering

Timeline

1.0 Project Initiation and Planning

- 1.1 Define goals & objectives
- 1.2 Identify Stakeholders
- 1.3 Planning & feasibility study

2.0 Requirements Gathering and Analysis

- 2.1 Gather requirements
- 2.2 Analyse & document requirements
- 2.3 Validate functional & non-functional requirements

3.0 First Panel Presentation

- 3.1 Gather & validate problem and objectives
- 3.2 Competitors analysis
- 3.3 Validate hardware & software requirements
- 3.4 Validate high-level system analysis

4.0 Software System Design

- 4.1 System architecture design
- 4.2 Database Design

- 4.3 Schema Design
- 4.4 Detailed component design
- 4.5 Interface design
- 4.6 Wireframes

5.0 System Implementation and Development

- **5.1 Backend Development**
  - 5.1.1 Data processing module
  - 5.1.2 Sentiment Analysis module
  - 5.1.3 Storage module
- **5.2 Frontend Development**
  - 5.2.1 User Interface
  - 5.2.2 Dashboard Development
- 5.3 API Integration

6.0 Testing and Validation

- 6.1 Component testing
- 6.2 NLP Module testing
- 6.3 Data processing testing
- 6.4 API testing
- 6.5 Sentiment Analysis accuracy testing
- 6.6 Load testing
- 6.7 User acceptance testing
- 6.8 Dashboard functionality

7.0 Deployment and Launch

- 7.1 Deployment planning

- 7.2 Setup & execute deployment
- 7.3 Configure monitoring tools
- 7.4 Run security checks
- 7.5 Finalize & launch

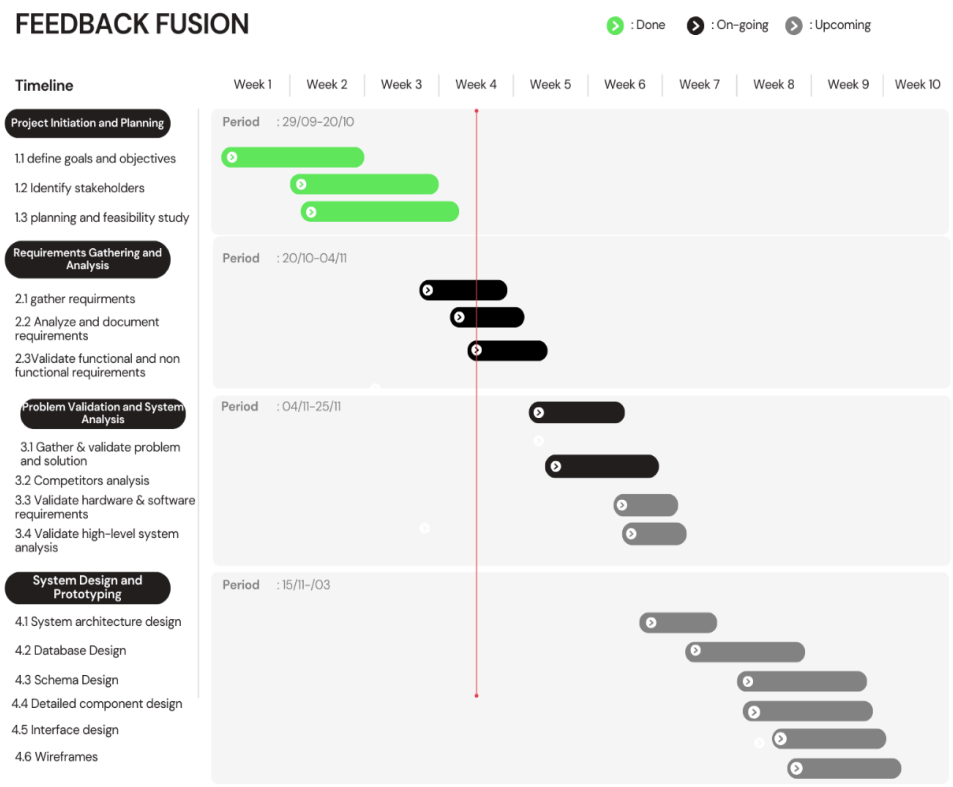
8.0 User Training and Documentation

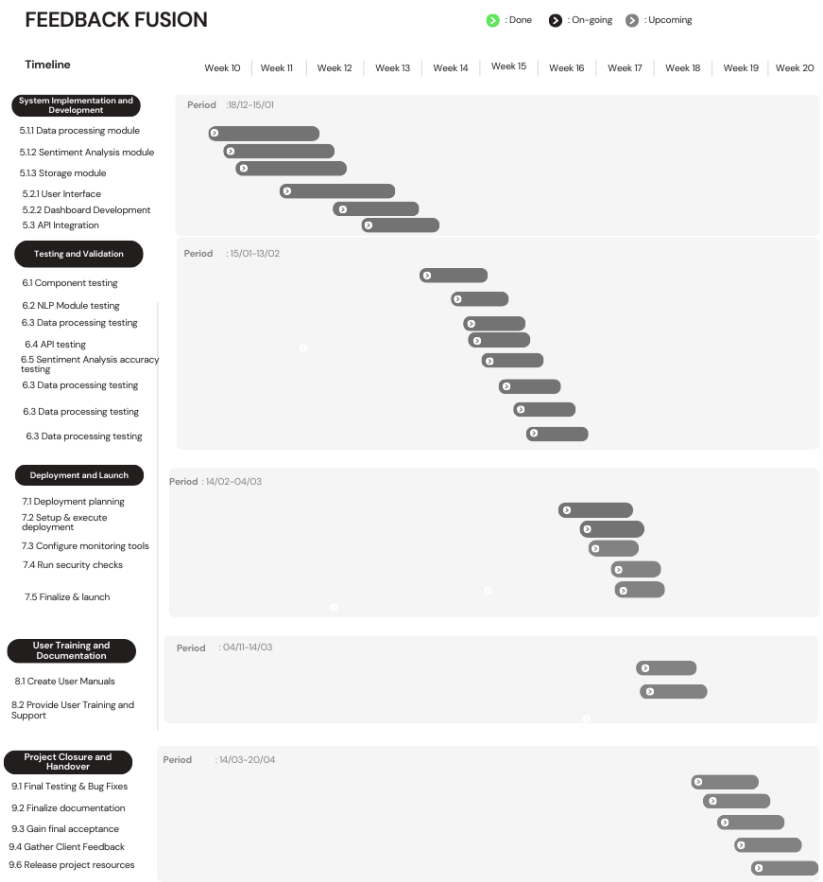
- 8.1 Create User Manuals
- 8.2 Provide User Training and Support

9.0 Project Closure and Handover

- 9.1 Final Testing & Bug Fixes
- 9.2 Finalize documentation
- 9.3 Gain final acceptance
- 9.4 Gather Client Feedback
- 9.5 Release project resources

Gantt Chart


FEEDBACK FUSION
: Done
: On-going
: Upcoming
Timeline
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Week 8
Week 9
Week 10
Project Initiation and Planning
1.1 define goals and objectives
1.2 Identify stakeholders
1.3 planning and feasibility study
Period : 29/09-20/10
Requirements Gathering and Analysis
2.1 gather requirments
2.2 Analyze and document requirements
2.3Validate functional and non functional requirements
Period : 20/10-04/11
Problem Validation and System Analysis
3.1 Gather & validate problem and solution
3.2 Competitors analysis
3.3 Validate hardware & software requirements
3.4 Validate high-level system analysis
Period : 04/11-25/11
System Design and Prototyping
4.1 System architecture design
4.2 Database Design
4.3 Schema Design
4.4 Detailed component design
4.5 Interface design
4.6 Wireframes
Period : 15/11-/03


FEEDBACK FUSION
: Done
: On-going
: Upcoming
Timeline
Week 10
Week 11
Week 12
Week 13
Week 14
Week 15
Week 16
Week 17
Week 18
Week 19
Week 20
System Implementation and Development
5.1.1 Data processing module
5.1.2 Sentiment Analysis module
5.1.3 Storage module
5.2.1 User Interface
5.2.2 Dashboard Development
5.3 API Integration
Period :18/12-15/01
Testing and Validation
6.1 Component testing
6.2 NLP Module testing
6.3 Data processing testing
6.4 API testing
6.5 Sentiment Analysis accuracy testing
6.3 Data processing testing
6.3 Data processing testing
6.3 Data processing testing
Period : 15/01-13/02
Deployment and Launch
7.1 Deployment planning
7.2 Setup & execute deployment
7.3 Configure monitoring tools
7.4 Run security checks
7.5 Finalize & launch
Period : 14/02-04/03
User Training and Documentation
8.1 Create User Manuals
8.2 Provide User Training and Support
Period : 04/11-14/03
Project Closure and Handover
9.1 Final Testing & Bug Fixes
9.2 Finalize documentation
9.3 Gain final acceptance
9.4 Gather Client Feedback
9.6 Release project resources
Period : 14/03-20/04

Implementation Details

Backend Implementation

The FeedbackFusion backend is built on Node.js with Express, providing a RESTful API that handles data processing, storage, and business logic:

Core Backend Components:

1. Express Application: Main server application handling HTTP requests and routing
2. Mongoose Models: Data models and database interaction
3. Controllers: Request handling and response generation
4. Middleware: Authentication, validation, and error handling
5. Services: Business logic and cross-cutting concerns

Key Backend Features:

1. Authentication System:
    - JWT-based authentication
    - Role-based access control
    - Session management
    - Password hashing with bcrypt
1. Data Processing Pipeline:
    - File upload handling with multer
    - Data validation and transformation
    - Integration with NLP service
    - Background processing for intensive tasks
1. Apify Integration:
    - Google Reviews scraping
    - Data normalization

    - Duplicate detection
    - Error handling
1. Alert System:
    - Rule-based monitoring
    - Threshold evaluation
    - User notification
    - Alert lifecycle management
1. Reporting Engine:
    - Data aggregation
    - Format transformation
    - File generation
    - Delivery mechanisms

API Structure:

- RESTful endpoints following resource-based naming
- Consistent response formats with appropriate HTTP status codes
- Pagination for large data sets
- Filtering and sorting capabilities
- Comprehensive error handling

Performance Optimizations:

- Database indexing for frequent queries
- Query optimization with projection and selection
- Caching strategies for frequently accessed data
- Background processing for long-running tasks
- Connection pooling for database efficiency

Frontend Implementation

The FeedbackFusion frontend is built with React, providing a responsive and interactive user interface:
Core Frontend Components:

1. React Application: Single-page application with component-based architecture
2. React Router: Client-side routing and navigation
3. Context API: State management for application-wide data
4. Recharts: Chart and visualization components
5. Tailwind CSS: Utility-first styling approach

Key Frontend Features:

1. Dashboard Components:
    - Metric cards with summary statistics
    - Interactive charts and graphs
    - Recent activity feed
    - Alert notifications
1. Data Visualization:
    - Line charts for trend analysis
    - Bar charts for comparative data
    - Pie charts for distribution analysis
    - Area charts for time series data
    - Custom visualizations for specific insights
1. Interactive Elements:
    - Filterable data tables
    - Expandable detail views
    - Search functionality
    - Drag-and-drop file upload
    - Interactive form elements

1. Responsive Design:
    - Web-first approach
    - Flexible layouts with CSS Grid and Flexbox
    - Breakpoint-specific optimizations
    - Touch-friendly controls for mobile devices

State Management:

- React Context API for application-wide state
- Local component state for UI-specific concerns
- Optimistic updates for improved user experience
- Efficient re-rendering through memoization and careful state structure

Performance Considerations:

- Code splitting for reduced initial load time
- Lazy loading of non-critical components
- Image optimization for visual elements
- Virtualization for long lists
- Debounced inputs for search and filtering

NLP Pipeline

The Natural Language Processing (NLP) pipeline is a critical component of FeedbackFusion, enabling automated analysis of feedback text:
Fine-Tuned Sentiment Analysis Model:

- Model Architecture: Custom fine-tuned sentiment analysis model based on DistilBERT
- Training Technology: Transformers library by Hugging Face, PyTorch for model training
- Training Dataset: 500,000 hotel-specific feedback data with labeled sentiment
- Fine-Tuning Process: Model was fine-tuned on hotel-specific language and expressions to improve accuracy for hospitality domain
- Performance: Achieves higher accuracy (86%) on hotel feedback compared to generic sentiment models

- Training Parameters:
    - Learning Rate: 2e-5
    - Batch Size: 32
    - Epochs: 2
    - Optimizer: AdamW with weight decay
- Capabilities:
    - Multi-class sentiment classification (very positive, positive, neutral, negative, very negative)
    - Confidence scoring for reliability assessment
    - Robust handling of hospitality-specific terminology
    - Automatically detects and translates feedback into English before analysis, enabling consistent sentiment evaluation across global guests.

NLP Service Architecture:

1. Flask API: Python-based service exposing NLP functionality via RESTful API
2. Preprocessing: Text cleaning, normalization, and tokenization
3. Model Inference: Application of fine-tuned models to input text
4. Post-processing: Formatting and enhancement of model outputs

NLP Pipeline Workflow:

1. Feedback text is sent to the NLP service from the main application
2. Text undergoes preprocessing to standardize format
3. The fine-tuned sentiment model classifies the overall sentiment
4. Emotion detection model identifies emotional content
5. Department classification is performed using zero-shot classification
6. A rule-based recommendation engine scans the text for negative keywords and assigns actionable recommendations accordingly
7. Results are returned to the main application and stored with the feedback

Integration Points:

- RESTful API between main application and NLP service

- JSON format for data exchange
- Error handling and fallback mechanisms
- Asynchronous processing for batch operations

Additional NLP Features:

- Emotion detection using distilRoBERTa model
- Department classification with zero-shot learning
- Entity recognition for identifying specific aspects of service
- Keyword extraction for topic modeling

Integration Points

FeedbackFusion integrates with several external systems and services to enhance its capabilities:
Apify Integration:

- Purpose: Automated scraping of Google Reviews
- Integration Type: API-based using Apify client library
- Authentication: API key
- Data Flow: Scheduled scraping jobs retrieve recent reviews, which are then processed and stored in the database

MongoDB Database:

- Purpose: Primary data storage
- Integration Type: Mongoose ODM
- Connection: Direct connection with connection pooling
- Data Flow: All application data is stored and retrieved from MongoDB

NLP Service:

- Purpose: Text analysis for sentiment, emotion, and classification

- Integration Type: RESTful API
- Communication: HTTP/HTTPS with JSON payloads
- Data Flow: Feedback text is sent to the service, which returns analysis results

Export Services:

- Purpose: Generation of reports in various formats (PDF & Excel)
- Integration Type: Client-side libraries (jsPDF, XLSX)
- Operation: Client-side generation of reports based on data retrieved from the backend
- Data Flow: Application data is transformed into downloadable file formats

Authentication System:

- Purpose: User authentication and authorization
- Integration Type: JWT-based token system
- Security: HTTPS for secure transmission, hashed passwords
- Data Flow: Credentials are verified and tokens are issued for authenticated sessions

Testing

Test Strategy and Approach

The testing strategy for FeedbackFusion focused on ensuring functionality, usability, and reliability across all system components:

Testing Approach:

1. Functional Testing: Manual execution of test cases covering all major features
2. Integration Testing: Verification of component interactions and data flow
3. User Acceptance Testing: Evaluation from the perspective of hotel staff

4. Cross-Browser Testing: Verification across different browsers and devices

Testing Methodology:

- Feature-Based Testing: Each system feature is tested in isolation
- Scenario-Based Testing: End-to-end workflows are tested to ensure complete functionality
- Edge Case Testing: Testing of unusual inputs and boundary conditions
- Regression Testing: Re-testing of existing features after changes

Test Data:

- Mixture of real and synthetic hotel reviews
- Varied sentiment, emotion, and content
- Multiple file formats for upload testing
- Data with edge cases and special characters

Test Environment

The testing environment for FeedbackFusion consisted of:
Hardware/OS:

- Development machines: MacBook Pro (macOS) and Windows 10 PC
- Testing conducted on multiple device types (desktop, laptop, tablet)

Software Configuration:

- Node.js: LTS version (v14+)
- MongoDB: v4.4+ (local instances for testing)
- NLP Service: Python Flask microservice running locally
- Browsers: Chrome, Firefox, Safari, Edge

Test Data Configuration:

- Pre-populated MongoDB database with test data
- CSV and JSON files for upload testing
- Mock responses for external service tests

Test Cases and Results

Below is a summary of key test cases executed for FeedbackFusion:

User Access and Management

Test ID	Feature	Description	Result
TC-001	Login	Verify login with valid/invalid credentials for different roles	Pass
TC-002	Logout	Verify session termination and redirect	Pass
TC-003	Add User	Verify admin can add new users to the system	Pass
TC-004	Remove User	Verify admin can remove users from the system	Pass
TC-005	Change Password	Verify users can change their passwords	Pass
TC-006	Grant Roles	Verify role assignment and permission enforcement	Pass

Data Collection

Test ID	Feature	Description	Result
TC-007	External Scraping	Verify Google Reviews scraping via Apify	Pass
TC-008	File Upload	Verify CSV/JSON upload and processing	Pass

NLP Pipeline

Test ID	Feature	Description	Result
TC-009	Sentiment Analysis	Verify sentiment classification accuracy	Pass
TC-010	Emotion Detection	Verify emotion classification accuracy	Pass

Alert System

Test ID	Feature	Description	Result
TC-011	Alert System	Verify alert creation and triggering	Pass

Historical Analysis

Test ID	Feature	Description	Result
TC-012	Historical Analysis	Verify trend visualization and filtering	Pass

Results

Test ID	Feature	Description	Result
TC-013	Generate Report	Verify report generation and export	Pass
TC-014	Filtering	Verify filtering by emotion, department, date	Pass
TC-015	Search	Verify keyword search functionality	Pass

Recommendation Engine

Test ID	Feature	Description	Result
TC-016	Recommendations	Verify rule-based recommendation generation	Pass

Test Results Summary:

- Pass Rate: 16 out of 16 test cases passed (100%)
- Critical Issues: None identified in implemented features
- Minor Issues: Some UI responsiveness issues on very small screens were addressed

Challenges and Solutions

Technical Challenges

NLP Accuracy and Performance:

- Challenge: Achieving reliable sentiment analysis for hotel-specific terminology and context.
- Solution: Created a fine-tuned sentiment analysis model specifically trained on hotel feedback data, significantly improving accuracy for domain-specific language and expressions. The model was optimized for both accuracy and performance to ensure timely processing of feedback.

Data Integration from Multiple Sources:

- Challenge: Handling different data formats and structures from various feedback sources.
- Solution: Implemented a flexible data processing pipeline with standardized intermediate representation and created specific adapters for each data source (CSV, JSON, Google Reviews) that transform data into a consistent internal format before processing.

Real-time Dashboard Performance:

- Challenge: Maintaining responsive dashboard performance with large datasets and complex visualizations.
- Solution: Implemented data aggregation at the database level, pagination for large result sets, and optimized React components to minimize re-renders. Used memoization techniques for expensive calculations and implemented efficient state management.

Database Query Optimization:

- Challenge: Ensuring efficient database queries for complex filtering and aggregation operations.
- Solution: Created appropriate indexes for frequently queried fields, used projection to limit returned fields, and implemented query caching for common operations. Restructured some queries to take advantage of MongoDB's aggregation pipeline for better performance.

Design Challenges

Balancing Feature Richness with Usability:

- Challenge: Providing comprehensive analytics capabilities without overwhelming users with complexity.
- Solution: Implemented a progressive disclosure approach where basic information is immediately visible with options to explore deeper. Created role-specific views that present relevant information based on user needs and responsibilities.

Responsive Design for Various Devices:

- Challenge: Creating a responsive interface that works well across desktop, tablet, and potentially mobile devices.
- Solution: Adopted a mobile-first approach with Tailwind CSS, ensuring layouts adapt gracefully to different screen sizes. Simplified complex visualizations on smaller screens and prioritized essential information for mobile users.

Data Visualization Clarity:

- Challenge: Representing complex data relationships and trends in an intuitive manner.
- Solution: Selected appropriate chart types for different data relationships, used consistent color coding for sentiment and emotions, and provided interactive elements (tooltips, filters) to explore data. Added context and explanations for complex visualizations to guide interpretation.

Alert System Design:

- Challenge: Creating an alert system that provides timely notifications without overwhelming users.
- Solution: Implemented priority levels for alerts, allowing users to focus on critical issues. Designed a customizable threshold system that lets hotels set their own criteria for what constitutes alert-worthy feedback. Created an intuitive interface for managing and dismissing alerts.

Solutions Applied

Modular Architecture Implementation:

- Challenge: Creating a system that allows for independent development and maintenance of components.

- Solution: Adopted a clear separation of concerns with well-defined interfaces between modules. Implemented a service-oriented approach where features like NLP analysis, data collection, and visualization operate as separate but interconnected services, allowing for isolated development and testing.

Cross-Browser Compatibility:

- Challenge: Ensuring consistent functionality across different browsers and environments.
- Solution: Used polyfills and feature detection for browser-specific implementations, standardized CSS with Tailwind, and implemented thorough cross-browser testing protocols. Adopted progressive enhancement to ensure core functionality works even when advanced features might not be fully supported.

Security Implementation:

- Challenge: Protecting sensitive hotel and guest data across the application.
- Solution: Implemented comprehensive security measures including JWT-based authentication, role-based authorization for all API endpoints, input validation to prevent injection attacks, and secure password storage with bcrypt hashing. Added audit logging for sensitive operations to maintain accountability.

NLP Service Integration:

- Challenge: Creating a reliable connection between the main application and the NLP service.
- Solution: Designed a robust API interface with error handling, retry logic, and fallback mechanisms. Implemented asynchronous processing for batch operations to prevent blocking the main application flow. Created a monitoring system to detect and alert on NLP service issues.

Future Improvements

Planned Enhancements

Enhanced Machine Learning Models:

- Further refinement of the sentiment analysis and emotion model with additional hotel-specific training data
- Implementation of aspect-based sentiment analysis to identify sentiment toward specific hotel features (e.g., room cleanliness, staff friendliness)
- Topic modeling to automatically identify emerging themes in feedback

Advanced Visualization Capabilities:

- Interactive dashboards with drill-down capabilities for deeper insights
- Customizable dashboard layouts for different user roles and preferences
- Advanced comparison tools for benchmarking against industry standards
- Geospatial visualization for multi-location hotels

Expanded Data Collection:

- Additional integrations with popular booking platforms (Booking.com, Expedia, etc.)
- Social media monitoring for brand mentions and feedback
- Direct integration with hotel PMS systems for guest data correlation
- Email and SMS survey capabilities with QR code generation

Alert and Notification System:

- Real-time notifications via multiple channels (email, SMS, etc.)
- Customizable alert rules with more complex conditions
- Escalation paths for unaddressed issues
- Automated suggestion of response templates based on feedback content

Report Generation Enhancements:

- Additional report templates for different stakeholder needs
- Scheduled report generation and distribution
- Interactive report elements for dynamic exploration
- Benchmark reporting against historical performance

User Experience Improvements:

- Mobile application for on-the-go access to critical alerts and insights
- Voice-based interaction for hands-free operation
- Accessibility enhancements for users with disabilities
- Improved onboarding and contextual help systems

Scalability Considerations

As FeedbackFusion evolves, several scalability factors have been identified for future development:

Technical Scalability:

- Database sharding for handling larger datasets while maintaining performance
- Microservices architecture to allow independent scaling of system components
- Containerization with Docker and orchestration with Kubernetes for flexible deployment
- Caching layers to improve performance for frequently accessed data

Organizational Scalability:

- Multi-property support for hotel chains and groups
- Role customization for more granular access control
- Team collaboration features for coordinated response to feedback
- Workflow automation for routing and responding to feedback

Feature Scalability:

- API development for third-party integrations
- Plugin architecture for custom extensions
- White-labeling options for integration with hotel branding
- Multi-tenant architecture for SaaS deployment

Conclusion

Project Outcomes

FeedbackFusion has successfully delivered a comprehensive hotel feedback management system that addresses the core challenges of collecting, analyzing, and deriving insights from guest feedback. The key achievements include:

1. Centralized Feedback Repository: The system successfully aggregates feedback from multiple sources, including file uploads and automated Google Reviews scraping, providing hotels with a single view of guest sentiment.
2. Intelligent Feedback Analysis: Leveraging a fine-tuned NLP model, FeedbackFusion accurately determines sentiment, emotion, and departmental relevance of feedback, transforming unstructured text into structured, actionable data.
3. Intuitive Visualization: The system presents complex feedback data through clear, interactive visualizations that enable hotel staff to quickly identify trends, issues, and opportunities for improvement.
4. Historical Analysis: The robust trend analysis capabilities allow hotels to track performance over time, measure the impact of service improvements, and identify seasonal patterns in guest satisfaction.
5. Role-Based Access: The implementation of role-specific views and permissions ensures that staff at different levels have appropriate access to feedback insights relevant to their responsibilities.
6. Alert System: The customizable alert system ensures that critical feedback is promptly identified and brought to the attention of relevant staff members.
7. Recommendation Engine: The rule-based recommendation system provides actionable suggestions based on feedback patterns, helping hotels prioritize improvement initiatives.

The core functionality provides significant value for hotel feedback management. The modular architecture ensures that these features can be enhanced in future iterations without disrupting existing capabilities.

Lessons Learned

The development of FeedbackFusion yielded several valuable insights:

Technical Lessons:

- Domain-specific sentiment analysis requires significant customization. The investment in fine-tuning a hotel-specific model proved essential for accurate results.
- Integrating diverse data sources requires robust normalization and validation processes to ensure consistency in analysis.
- Early attention to database indexing and query optimization is crucial for maintaining responsive performance as data volumes grow.

Project Management Lessons:

- Focusing on core features first (data collection, analysis, visualization) allowed for earlier delivery of value while maintaining flexibility for feature adjustments.
- The decision to implement a modular architecture paid dividends in allowing parallel development and easier testing of individual components.
- Early user testing helped refine the interface and feature set to better align with actual hotel staff needs and workflows.

Domain-Specific Insights:

- Analysis revealed common patterns in hotel feedback that informed both the NLP model training and the development of the recommendation rules.
- The importance of departmental classification became evident as different hotel areas require different response strategies and staff involvement.
- Hotel managers showed a strong preference for trend-based visualizations that highlight changes over time rather than just current status.

FeedbackFusion represents a significant step forward in hotel feedback management, providing a foundation for data-driven decision making and service improvement. The system's modular design and extensible architecture ensure it can evolve to meet changing hotel needs and incorporate additional features in future releases.

References

1. Anon, (2024). Turning Reviews into Revenue: The Impact of Guest Feedback on Hotels. [online] Available at: https://www.onressoftware.com/impact-of-guest-feedback-on-hotel-performance/.
2. Nikhil (2021). Guide to Sentiment Analysis using Natural Language Processing. [online] Analytics Vidhya. Available at: https://www.analyticsvidhya.com/blog/2021/06/nlp-sentiment-analysis/#h-sentiment-analysis-challenges.
3. Ali, M. (2023). NLTK Sentiment Analysis Tutorial: Text Mining & Analysis in Python. [online] www.datacamp.com. Available at: https://www.datacamp.com/tutorial/text-analytics-beginners-nltk.
4. Webuters (n.d.). Historical Data: How It Helps In Business Decision Making? - Webuters. [online] www.webuters.com. Available at: https://www.webuters.com/historical-data-in-business-decision-making.
5. www.domo.com. (n.d.). Domo Resource - How API integration can enhance data analysis and reporting. [online] Available at: https://www.domo.com/learn/article/how-api-integration-can-enhance-data-analysis-and-reporting.
6. Anon, (2022). Why is Sentiment Analysis of Hotel Reviews important? [online] Available at: https://10xds.com/blog/sentiment-analysis-of-hotel-reviews/.
7. GeeksforGeeks (n.d.) Use Case Diagram. Available at: https://www.geeksforgeeks.org/use-case-diagram/
8. MongoDB Documentation (2024). MongoDB Node.js Driver. [online] Available at: https://docs.mongodb.com/drivers/node/current/
9. React.js Documentation (2024). React: A JavaScript library for building user interfaces. [online] Available at: https://reactjs.org/docs/getting-started.html
10. Hugging Face (2023). Transformers Documentation. [online] Available at: https://huggingface.co/docs/transformers/index